# Efficient Mamba-Based Accelerator Architecture for Massive-MIMO Indoor Localization Using the LuViRA Dataset

Shengjie Chen
sh5704ch-s@student.lu.se

Chenxi Zhang
ch7210zh-s@student.lu.se

Department of Electrical and Information Technology
Lund University

Supervisor: Ilayda Yaman
Sijia Cheng

Examiner: Pietro Andreani

May 31, 2026

© 2026
Printed in Sweden
Tryckeriet i E-huset, Lund

# Acknowledgments

We would like to express our sincere gratitude to our supervisor, Ilayda Yaman, for her continuous guidance, support, and feedback throughout this thesis work. Her advice helped us define the research direction, improve the structure of the thesis, and better understand how the proposed model could be connected with practical indoor localization applications. Working with her has been a valuable experience during our master's studies.

We would also like to thank our second supervisor, Sijia Cheng, for her helpful technical discussions and suggestions during the project. Her feedback was especially valuable in improving the hardware implementation and evaluating the FPGA accelerator.

We are sincerely grateful to Professor Liang Liu for his valuable guidance and support throughout this project. We greatly appreciate his involvement in the supervision process and his constructive feedback during the thesis review and presentation preparation stages. His academic insights and suggestions were highly valuable to the completion of this work.

We are also grateful to the Department of Electrical and Information Technology at Lund University for providing the academic environment and resources needed to carry out this thesis. We also thank the researchers and engineers whose previous work on Massive MIMO indoor localization, the LuViRA dataset, and FPGA acceleration provided an important foundation for our study.

Finally, we would like to thank our families and friends for their patience, encouragement, and support throughout this period.

*Shengjie Chen and Chenxi Zhang*
*May 2026*

# Abstract

In recent years, as 6G wireless systems continue to develop, high-accuracy indoor localization has become increasingly important for for future wireless systems. At the same time, reliable indoor positioning is needed by many location-aware applications, such as robotic navigation, emergency healthcare, and smart transportation. Since indoor wireless measurements can vary over time due to multipath propagation and environmental changes, short sequences of observations may provide useful information for position estimation. Mamba, a sequence model based on selective state-space updates, is therefore a promising candidate for this task because it can jointly process short sequences of wireless observations and capture temporal variations in the channel.

This thesis investigates a hardware-friendly Slim-Mamba approach for Massive-MIMO indoor localization using the LuViRA dataset. Software evaluation is first used to select a simplified Mamba-style model that preserves the recurrent update and gating behavior needed for localization while reducing the complexity of larger channel-fusion variants. A Zynq MPSoC-based FPGA accelerator is then designed, with processor-side control, programmable-logic input loading, four-block Slim-Mamba computation, and AXI-stream dataflow. We designed a shared $4 \times 4 \times 4$ reconfigurable MAC fabric for linear projection layers, together with a fine-grained FIFO-based state-space model (SSM) pipeline to reduce waiting time between projection, nonlinear, and state-update stages. A fixed-point quantization flow is built from Python, to bit-true C++ modeling, and finally to RTL.

On our software localization evaluation, the floating-point Slim-Mamba model achieves a mean localization error of 0.13500 m, while the fixed-point hardware model achieves 0.14103 m with an output MAE of 0.02081 against the floating-point output. The FPGA design closes timing at 100 MHz and reaches an estimated throughput of 471 frames/s, exceeding the 100 frames/s real-time target. The post-synthesis on-chip power estimate is 4.292 W, where processor-side control and data movement dominate the dynamic power; the programmable-logic accelerator itself accounts for about 0.94 W of dynamic power. An on-board validation test further verifies that processor control, memory movement, streaming interfaces, and FPGA computation operate together as a complete prototype system.

# Popular Science Summary

Positioning a person or device is often supported by satellite navigation in open outdoor environments. However, satellite-based positioning can become unreliable when signals are blocked or reflected by buildings, trees, or dense urban surroundings. Indoors, this problem is usually even more severe, because satellite signals are often weak or unavailable. Still, many future applications need reliable indoor positioning: a robot may need to move safely in a building, a hospital may need to locate equipment, and a smart environment may need to understand where people or devices are.

This thesis studies how wireless signals can be used for this purpose. When a radio signal travels through a room, it bounces off walls, furniture, and people before it reaches the receiver. These reflections create a wireless "fingerprint" of the environment. If we can learn how these fingerprints relate to positions, then a wireless system can estimate where a device is located.

Machine learning is useful for this task because it can learn patterns that are too complicated to describe by hand. In this work, we study a modern type of sequence model called Mamba. A simple way to think about it is that the model does not look at one measurement alone. Instead, it can look at a short sequence of recent measurements and remember useful changes over time. This is promising for indoor positioning.

However, a model that works well on a computer is not automatically suitable for small and efficient hardware. Therefore, this work builds a hardware accelerator for this slim model on an FPGA, which is a programmable chip. The accelerator can be seen as a small custom factory: data enters from memory, passes through several processing stages, and the final position-related output is written back. To make the factory efficient, the same calculation units are reused for several parts of the model, and the most time-critical steps are arranged like an assembly line so that one step can start before the previous one has completely finished.

# Table of Contents

# List of Figures

# List of Tables

# Chapter 1
# Introduction

Indoor localization is becoming an important feature for future 6G wireless systems, since 6G is expected to support not only ubiquitous communication but also high-accuracy localization and high-resolution sensing services [3]. Localization is also regarded as a key enabler for future 6G wireless systems [4]. With the development of location-aware applications such as robotic navigation, emergency healthcare, and smart transportation, wireless localization is expected to support a wider range of location-aware services [1]. At the same time, real-time indoor positioning has long remained a challenging problem [5]. In indoor wireless localization systems, localization performance is affected by environmental factors such as multipath and non-line-of-sight propagation, which makes accurate positioning challenging [6].

Machine-learning-based radio localization provides a data-driven way to learn the relationship between wireless measurements and user positions. In Massive-MIMO-based indoor localization, rich radio channel observations can be used as location-dependent ML features, and previous work has demonstrated centimeter-level localization accuracy using measured indoor Massive MIMO data [1, 7]. However, practical deployment still faces challenges. High-dimensional radio features can lead to large neural networks, high memory demand, and increased training and inference cost. These issues become more critical when the localization model needs to be implemented on dedicated hardware with limited computation resources, memory capacity, and power budget.

This motivates the study of compact and hardware-friendly neural architectures for Massive MIMO indoor localization. Mamba [2], proposed as a recent sequence model based on recurrent state-space updates, is attractive in this context because it replaces several more complex neural-network components with a relatively compact recurrent state-space model (SSM) backbone. In this thesis, Mamba-family models are investigated as a possible alternative to large fully connected localization models, with the aim of studying the trade-off among localization accuracy, model size, and hardware implementation complexity.

## 1.1 Related Work

In recent years, machine learning has been investigated for Massive-MIMO-based indoor localization. Tian et al. proposed fully connected neural net-

work (FCNN)-based localization methods that use spatial covariance matrices and truncated channel impulse responses to capture angular- and delay-domain information, achieving centimeter-level accuracy in indoor Massive MIMO measurements [7]. Their later work further introduced an ML-based localization pipeline with multiple processing chains and ensemble learning to fuse different channel fingerprints. However, these studies also show that although subarray-based processing can reduce network size, efficient and hardware-friendly model design remains an important issue for practical deployment [1].

Most existing ML-based Massive MIMO indoor localization methods rely on fully connected neural networks (FCNNs), convolutional neural networks (CNNs) [8], k-nearest neighbors (KNN) [9], or other conventional machine-learning models to process high-dimensional channel fingerprints [1]. These methods can effectively exploit channel features, but for densely connected models such as FCNNs, higher-dimensional inputs usually lead to larger network sizes, more parameters, and higher training and inference cost. Existing work also points out that, when ML-based localization methods are applied to Massive MIMO systems, the increase in network size can make training and fine-tuning more challenging [1]. Therefore, reducing model size, computational complexity, and hardware deployment cost while maintaining localization accuracy remains an important research problem in Massive-MIMO-based indoor localization.

Mamba was originally proposed by Gu and Dao as a selective state-space sequence model [2]. Different from attention-based architectures, Mamba processes sequences through recurrent state updates and makes part of the state-space model parameters input-dependent. This allows the model to selectively propagate or forget information according to the current input, while maintaining linear scaling with sequence length [2]. The original Mamba work also emphasizes a hardware-aware implementation of the selective scan, making the architecture attractive not only from an algorithmic perspective but also from a hardware computation-structure perspective.

For Massive MIMO indoor localization, Mamba is worth investigating because the radio input can be organized as short sequences or tokenized channel features, and the recurrent state update may provide a compact way to integrate useful contextual information. Also, the original Mamba architecture was not designed specifically for radio localization or FPGA deployment, which motivates the development of a task-driven Slim-Mamba variant and a corresponding accelerator design for the target localization workload.

## 1.2 Goals and Challenges

The goal of this thesis is to investigate the feasibility and effectiveness of Mamba-like architectures for Massive MIMO indoor localization, with a particular focus on localization accuracy, model size, and hardware implementation complexity. In addition, this thesis further explores FPGA-based acceleration of the proposed Mamba-like architecture.

Compared with previous FCNN-based methods that mainly operate on single snapshots or fixed fingerprints, Mamba can process short sequences of observations

and capture temporal structure in the wireless channel. This thesis aims to evaluate Mamba-family models for LuViRA indoor localization, design a hardware-friendly Mamba-like architecture, and implement the key Mamba datapath on FPGA with quantization-aware considerations.

The main challenges include preserving localization accuracy after model simplification and quantization, handling the memory and computation pressure caused by high-dimensional channel state information (CSI), and balancing latency, power, resource utilization, and localization accuracy in the final accelerator design. Recent studies have explored Mamba acceleration in other application scenarios, including FPGA-based quantization and hardware co-design with computation reordering and fine-grained tiling, pipelined vector processing units, and hybrid dataflows with state-space buffers [10, 11, 12]. Motivated by these works, this thesis further investigates how the Slim-Mamba datapath can be reused for indoor localization, with particular attention to reconfigurable architechture, memory access organization, and pipeline granularity.

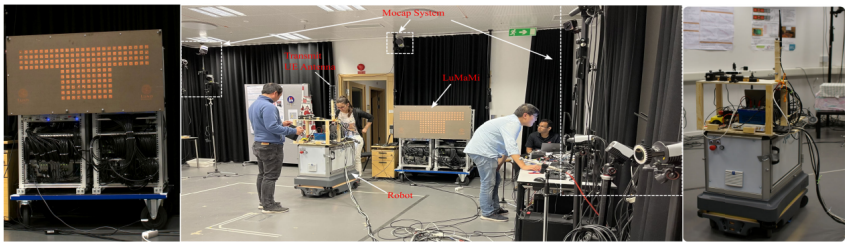
# Chapter 2
# Background

This section introduces the background needed for the rest of the thesis. The discussion is limited to three aspects: Massive MIMO as the radio measurement platform, radio-based positioning methods, and the LuViRA radio dataset used in this work.

## 2.1 Indoor Localization and Massive MIMO

Massive multiple-input multiple-output (MIMO) provides a promising measurement platform for radio-based indoor localization. By equipping the base station with a large antenna array, Massive MIMO provides rich spatial channel observations that can be exploited for localization. Previous work has used the instantaneous spatial covariance matrix to capture angular information and the channel impulse response to capture delay information [7]. These angular-domain and delay-domain features are useful because they provide complementary descriptions of the location-dependent radio channel in complex indoor environments. Therefore, Massive MIMO can provide favorable conditions for high-precision indoor positioning by enabling location-dependent channel information to be extracted from measured radio signals [7].

Figure 2.1 illustrates a representative Massive-MIMO-based indoor localization setup. A user equipment (UE) mounted on a mobile industrial robot moves inside the measurement area, while a base-station antenna array records the radio channel from different positions.




(a) BS LuMaMi (b) Measurement Scenario (c) UE on the robot

**Figure 2.1:** Massive MIMO indoor localization measurement setup with moving user equipment, adapted from [1].

Even though massive MIMO provides high-dimensional spatial channel measurements, the final positioning accuracy still depends on how the channel information is represented and how the localization model extracts useful features from it.

## 2.2 Radio-Based Positioning Methods

Radio localization estimates the position of a user equipment or mobile platform from measured wireless channel information. In the LuViRA dataset, the radio modality is described as the channel response between a 5G massive multiple-input multiple-output testbed and user equipment [13]. The localization system uses radio measurements whose values depend on the relative position between the transmitter, the receiver array, and the surrounding indoor environment. These measured channel responses can be used as location-dependent fingerprints for positioning.

Traditional localization methods mainly belong to the signal-processing-based family. Their main challenges are high algorithmic complexity and strict base-station (BS) array-calibration requirements [1]. In contrast, learning-based methods provide a more flexible data-driven framework in which different radio representations can be used as model inputs. In the LuViRA dataset, the radio measurements are collected in the same measurement campaign as the motion-capture ground truth. The channel responses are recorded by the Massive MIMO radio system, while the accurate pose labels are obtained from the motion capture system. These data streams are synchronized so that each radio observation can be paired with the corresponding ground-truth pose [13]. This yields paired radio measurements and position labels that can be used to train supervised fingerprinting or regression models for localization.

ML models take radio channel fingerprints as inputs and estimate the position of the user equipment. This is attractive for Massive MIMO indoor localization because the measured channel contains rich spatial and delay information that may be difficult to fully represent using explicit geometric methods. Previous work has also shown that multiple processing chains can be trained with different channel fingerprints and then combined to improve localization performance [1].

## 2.3 LuViRA Radio Dataset Used in This Thesis

The experimental data used in this thesis are taken from the radio modality of the Lund University Vision, Radio, and Audio (LuViRA) dataset. LuViRA is a synchronized multi-sensor dataset for indoor localization, including color images, depth maps, inertial measurement unit readings, radio channel responses, audio recordings, and accurate six-degree-of-freedom ground truth pose measurements [13]. Although the full dataset contains several sensing modalities, this thesis focuses only on the grid data from the radio measurements, since the target application is Massive MIMO indoor localization.

In the radio part of LuViRA, the measured data are described as the channel response between LuMaMi [14] and user equipment. In the measurement setup,

a transmit antenna is mounted on a slowly moving service robot together with the other sensors [13]. Therefore, the object localized in this thesis is the moving platform, represented by the radio transmitter mounted on the robot. The ground truth position is provided by the motion capture system, which is used to verify the localization results obtained from the sensor measurements.

The LuViRA dataset contains 89 recorded trajectories, and each trajectory includes approximately 20 to 50 seconds of synchronized sensor data and ground truth labels [13]. An example of the grid trajectory layout used in the LuViRA radio measurements is shown in Fig. 2.2. In this thesis, the radio grid data are used as model inputs, and the corresponding ground-truth positions are used as supervised labels during training. Following the FCNN-based localization baseline, the model is trained by minimizing the position loss between the predicted position and the ground-truth position [7]. For evaluation, the localization error of the $i$-th sample is defined as the Euclidean distance between the estimated position $\hat{\mathbf{p}}_i$ and the ground-truth position $\mathbf{p}_i$,

$$e_i = \|\hat{\mathbf{p}}_i - \mathbf{p}_i\|_2.$$

The reported localization accuracy is the mean localization error over all windowed test samples,

$$\bar{e}_{\text{test}} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} e_i,$$

where $N_{\text{test}}$ is the number of test windows after sequence construction.

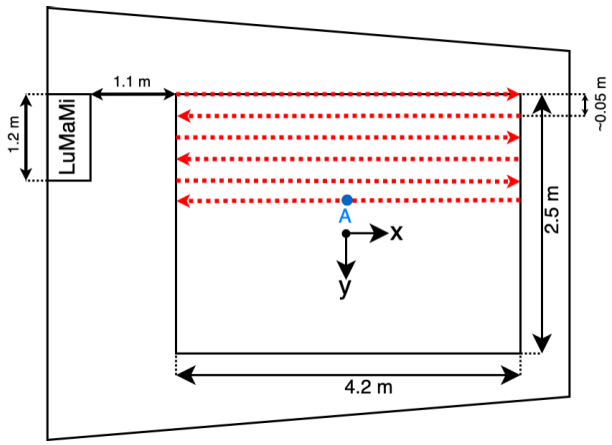



**Figure 2.2:** Example grid trajectory layout in the LuViRA radio dataset, adapted from [1].

The dataset used in this work is divided into three non-overlapping subsets: training, evaluation, and test. Following the same data split strategy as the FCNN baseline used for comparison [15], the test set and the training/evaluation pool are separated according to the parity of the grid index. The even-grid data are used as the test set, while the odd-grid data are used as the training and evaluation

pool. The evaluation set is then formed by randomly selecting two grids from the odd-grid data, and the remaining odd-grid data are used for training.

This split keeps the training, evaluation, and test sets separated at the grid level while following the same experimental setting as the FCNN baseline. As a result, the proposed Mamba-based models and the previous FCNN-based method can be compared under a consistent data partition, making differences in localization accuracy more attributable to the model design rather than to different dataset splits.

In this pipeline, one localization sample is constructed from a window of $K$ consecutive radio observations, represented as a $K \times D_{\text{in}}$ input matrix $\mathbf{X}_i$, together with the target position of the corresponding window endpoint. With the current setting used throughout the software and hardware flow, the input window contains the $K$ observations immediately preceding the target index. This definition of sequence length is used consistently in training, software evaluation, export, and FPGA-side reference inference.

# Chapter 3
# Algorithm

## 3.1 Mamba Basics

Mamba is a sequence modeling architecture based on selective state-space models (selective SSMs). A state-space model represents historical information using a hidden state and describes the dynamics of an input sequence through state transitions.

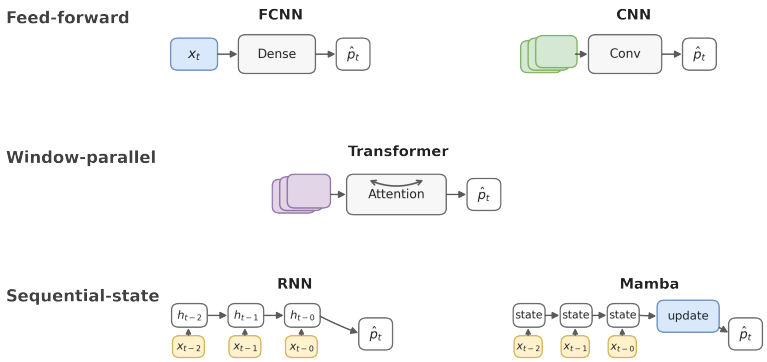



**Figure 3.1:** Comparison of temporal processing patterns in common localization backbones.

Figure 3.1 places Mamba in the context of other backbone families relevant to localization. FCNNs and CNNs mainly realize feed-forward mappings over a single snapshot or a fixed local window. RNNs and Mamba instead propagate an explicit state step by step, while Transformers jointly process all tokens within a window through attention. For this thesis, the distinction matters because it changes whether temporal information is captured by a fixed combinational mapping or by a sequential state update. In the present implementation, the sequence length $K$ denotes the number of consecutive radio observations in one input window. After channel projection and patch embedding with patch length $P$ and stride $S$, the recurrent backbone receives $L = (K - P)/S + 1$ tokens.

A continuous-time state-space model can be written as

$$\dot{h}(t) = Ah(t) + Bx(t), \tag{3.1}$$

$$y(t) = Ch(t) + Dx(t), \tag{3.2}$$

where $x(t)$ is the input, $h(t)$ is the hidden state, and $y(t)$ is the output. The matrix $A$ controls the state dynamics, $B$ determines how the input affects the state, $C$ maps the hidden state to the output, and $D$ represents a direct input-to-output path. To process discrete sequences, the continuous system can be discretized into a recurrent form:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \tag{3.3}$$

$$y_t = C_t h_t + Dx_t, \tag{3.4}$$

where $x_t$ is the input at time step $t$, $h_t$ is the current hidden state, and $y_t$ is the current output. The discretized parameters can be written as

$$\bar{A}_t = \exp(\Delta_t A), \tag{3.5}$$

$$\bar{B}_t = (\Delta_t A)^{-1} \left(\exp(\Delta_t A) - I\right) \Delta_t B_t. \tag{3.6}$$

The key idea of Mamba is to make part of the SSM parameters input-dependent. For example, $\Delta_t$, $B_t$, and $C_t$ can be generated from the current input $x_t$ through lightweight linear projections. A linear projection maps an input token to another feature space through an affine transformation:

$$\mathrm{Linear}_{W,b}(x_t) = Wx_t + b, \tag{3.7}$$

where $x_t$ is the input token, $W$ is a learnable weight matrix, and $b$ is a learnable bias vector. In the original Mamba model, input-dependent parameters such as $\Delta_t$, $B_t$, and $C_t$ are generated from the current input using such lightweight projections:

$$\tilde{\Delta}_t = W_\Delta x_t + b_\Delta, \quad B_t = W_B x_t + b_B, \quad C_t = W_C x_t + b_C. \tag{3.8}$$

The time-step parameter is then usually constrained to be positive by applying a nonlinear function such as softplus:

$$\Delta_t = \mathrm{softplus}(\tilde{\Delta}_t). \tag{3.9}$$

In this way, the model can dynamically decide what information should be preserved, updated, or forgotten according to the current input. Gu and Dao show that making SSM parameters functions of the input allows the model to selectively propagate or forget information along the sequence dimension, while Mamba maintains linear scaling with sequence length [2].

From an architectural perspective, a Mamba block usually consists of an input projection, local feature mixing, a selective scan, a gating branch, and an output projection. The input is first projected and split into a main branch and a gate branch. The main branch generates the dynamic SSM parameters and updates the hidden state through the selective scan. The gate branch is usually passed through a nonlinear activation function such as SiLU and then multiplied with the SSM output, followed by an output projection. Since the state update can be implemented recurrently over time, Mamba can model contextual information with

low sequence complexity and avoids the quadratic complexity commonly associated with attention-based methods on long sequences [2].

Mamba is worth investigating for indoor localization because indoor wireless measurements can often be organized as user trajectories or consecutive channel snapshots, where neighboring snapshots may contain short-term contextual information useful for position estimation. Compared with FCNN-based methods that mainly operate on single snapshots or fixed channel fingerprints, Mamba can progressively aggregate historical observations through its recurrent state and adaptively update the state according to the current input. Therefore, Mamba provides a promising model family that can exploit temporal context while maintaining linear sequence complexity, making it suitable for further investigation from the perspectives of accuracy, efficiency, and hardware implementation for Massive-MIMO-based indoor localization.

## 3.2 Slim-Mamba

In this work, Slim-Mamba is designed as a lightweight Mamba variant for Massive-MIMO-based indoor localization. The model is not obtained by directly adopting the original Mamba architecture, but rather through a task-driven simplification process based on preliminary model exploration. The initial design followed a channel-aware Mamba direction, where multi-branch structures were used to fuse different types of radio features, such as the real and imaginary parts of the channel impulse response (CIR), power features, and first-order difference features. A similar motivation can be found in CMamba, where Vanilla Mamba is considered insufficient for modeling cross-channel dependencies in multivariate time series, and additional channel-correlation modeling is introduced to enhance interactions among different channels [16]. For the LuViRA indoor localization task, different radio-domain features may also contain complementary location-related information, making channel-fusion design a reasonable initial direction.

However, preliminary experiments showed that the additional channel-fusion branches provided only limited accuracy improvement for the current localization task while significantly increasing model size, training time, and hardware mapping complexity. In addition, the Vanilla Mamba v1 baseline, implemented following the original Mamba architecture proposed by Gu and Dao [2], showed a clear generalization gap on LuViRA, whereas short temporal windows were already sufficient to provide useful localization information. Therefore, the final model follows a task-driven simplification principle: it preserves the recurrent state update and input-dependent modulation mechanisms that are most relevant to the localization task, while removing complex multi-branch fusion structures and the full selective-scan parameterization.

Figure 3.2 summarizes the block-level replacement from the original Mamba design to Slim-Mamba. The front-end and back-end structure is retained: both blocks use normalization, input projection and split, output gating, output projection, and a residual connection. The modification is concentrated in the state-update path. In the original Mamba block, the state branch generates input-dependent $\Delta_t$, $B_t$, and $C_t$, then uses $\Delta_t$ to discretize the continuous-time parame-

ters $A$ and $B$ before applying the selective scan recurrence [2]. Thus, $\Delta_t$ is not itself the recurrent update coefficient; it controls the conversion from the continuous-time SSM to the discrete-time recurrence through $(\bar{A}_t, \bar{B}_t) = \text{discretize}(\Delta_t, A, B)$. The resulting state is then mapped to the output through the explicit readout $C_t h_t + D\tilde{x}_t^{(s)}$.

Slim-Mamba replaces this sequence with a direct discrete-time update. The state branch generates a single input-dependent gate $\lambda_t$, and the state is updated as a convex combination of the previous state and the current state-branch input. Accordingly, $\lambda_t$ does not play the role of $\Delta_t$: it is not used for discretization, but enters the recurrence directly. The explicit $(C_t, D)$-based readout is also removed, so the recurrent state is sent directly to the output-gating path. Table 3.1 summarizes these substitutions in compact operator form, and Eqs. (3.10)– (3.15) define the final recurrence used in the remainder of this thesis.

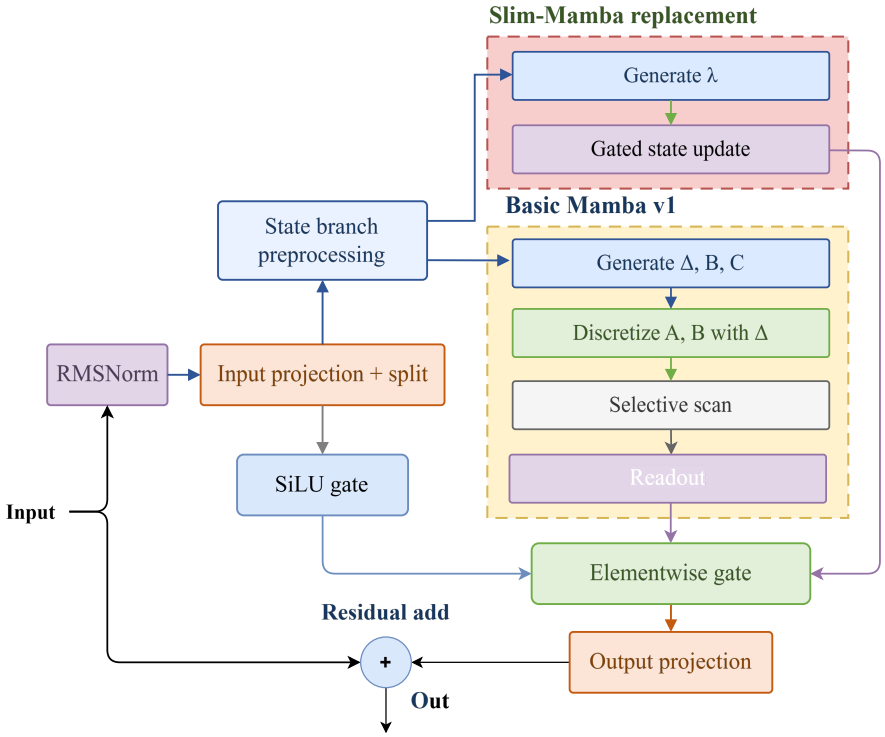



**Figure 3.2:** Original Mamba block and its Slim-Mamba replacement.

**Table 3.1:** Main operator substitutions between the original Mamba block [2] and the Slim-Mamba block used in this work.

| **Stage** | **Original Mamba [2]** | **Slim-Mamba in this work** |
|---|---|---|
| Block skeleton | RMSNorm → in-proj → split → gate → out-proj → residual | RMSNorm → in-proj → split → gate → out-proj → residual |
| State branch | Conv + SiLU | DWConv + SiLU |
| Parameter generation | $B_t,\ C_t,\ \Delta_t$ discretize$(\Delta_t, A, B)$ | $\lambda_t$ |
| State update | $h_t = \bar{A}_t h_{t-1} + \bar{B}_t \tilde{x}_t^{(s)}$ | $h_t = \lambda_t \odot h_{t-1} + (1-\lambda_t) \odot \tilde{x}_t^{(s)}$ |
| Readout | $r_t = C_t h_t + D\tilde{x}_t^{(s)}$ | $r_t = h_t$ |
| Output | $g_t = \mathrm{SiLU}(x_t^{(g)})$ $y_t = W_{\mathrm{out}}(r_t \odot g_t)$ | $g_t = \mathrm{SiLU}(x_t^{(g)})$ $y_t = W_{\mathrm{out}}(r_t \odot g_t)$ |

The deployed configuration used in the later hardware chapters is $D_{\mathrm{in}} = 2100$, $K = 2$, proj_dim $= 64$, $d_{\mathrm{model}} = 128$, $n_{\mathrm{layer}} = 4$, patch length $P = 2$, and stride $S = 2$. Table 3.2 breaks down the trainable parameters of this deployed path by function. Most parameters are concentrated in the repeated in-proj and *dt*-proj stages inside the four Slim-Mamba blocks, which is consistent with the parts later mapped to the main matrix-vector and state-update datapaths in hardware. The software module still contains an alternative pooled output head for ablation convenience, but that branch is not used in the deployed flat-head path and is therefore excluded from the table.

**Table 3.2:** Trainable parameter breakdown of the deployed Slim-Mamba configuration.

| **Component** | **Parameters** |
|---|---:|
| Input projection $D_{\mathrm{in}} \rightarrow$ proj_dim | 134,464 |
| Patch embedding Conv1d | 16,512 |
| Four block RMSNorm layers | 512 |
| Four block in-proj layers | 262,144 |
| Four block selective-scan *dt*-proj layers | 263,168 |
| Four block out-proj layers | 131,072 |
| Final RMSNorm | 128 |
| Flat output layer | 8,256 |
| Regression head proj_dim $\rightarrow 2$ | 130 |
| **Total** | **816,386** |

Table 3.3 summarizes the empirical comparison among the FCNN baseline, channel mamba, Vanilla Mamba v1, and Slim-Mamba on the LuViRA dataset. Compared with channel mamba, Slim-Mamba achieves a comparable test mean error while reducing the model size from 40.4 MB to 3.40 MB and the training time from 73 s to 13 s per epoch. Compared with Vanilla Mamba v1, Slim-Mamba

significantly improves localization accuracy while maintaining a similarly compact model size. These results indicate that Slim-Mamba provides a better trade-off among localization accuracy, model complexity, training efficiency, and hardware cost for the target indoor localization task.

**Table 3.3:** Comparison of localization accuracy and implementation cost across model variants.

| Model | Input Features | Size (MB) | Time (s/epoch) | it/s | Err. (cm) | HW Cost Proxy |
|---|---|---|---|---|---|---|
| FCNN[7] | CIR | 789 | N/A | N/A | ∼13 | High |
| channel mamba | CIR + Power | 40.4 | 73 | 13.18 | 12.9 | High |
| Vanilla Mamba v1 | CIR + Power | 3.38 | 33 | 32.62 | 24.74 | Medium |
| Slim-Mamba | CIR + Power | 3.40 | 13 | 75.30 | 13.5 | Low |

As illustrated in Fig. 3.3, the Slim-Mamba block first applies normalization to the input tokens, followed by a linear input projection that maps the input into an internal feature space. The projected representation is then split into two paths: a state input branch and a gate branch. The state input branch is used for recurrent state updating, while the gate branch is used for output modulation.

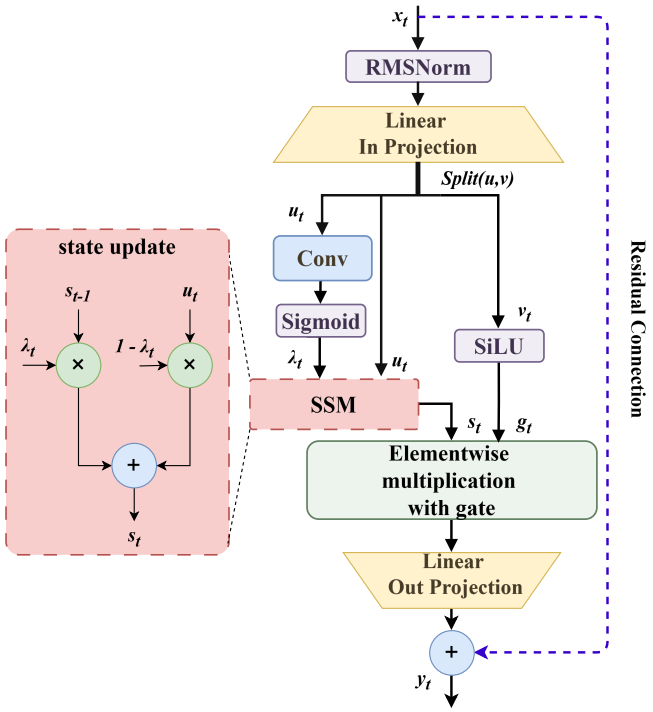



**Figure 3.3:** Simplified datapath of a Slim-Mamba block.

After the input projection and branch split, the token feature at time step $t$ is

denoted by $x_t^{(s)}$ on the state-update branch and by $x_t^{(g)}$ on the gate branch. In the state branch, the update gate generator produces an input-dependent coefficient $\lambda_t$ from the current input feature:

$$\lambda_t = \sigma(W_\lambda * x_t^{(s)} + b_\lambda), \tag{3.10}$$

where $*$ denotes the convolution operation, $\sigma(\cdot)$ is the sigmoid function, defined as

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \tag{3.11}$$

and $\lambda_t$ is constrained to the range $(0, 1)$ so that it can control the interpolation between the previous state and the current input. The recurrent state update is computed as

$$h_t = \lambda_t \odot h_{t-1} + (1 - \lambda_t) \odot x_t^{(s)}, \tag{3.12}$$

where $h_{t-1}$ is the previous state, $x_t^{(s)}$ is the current state-branch input, and $\lambda_t$ is the update gate that controls the balance between the historical state and the current input. When $\lambda_t$ is fixed, this recurrence is equivalent to an exponential moving average, since older inputs receive weights that decay as powers of $\lambda$. In Slim-Mamba, however, $\lambda_t$ is dynamically generated from the current input, and the module is therefore more accurately described as an input-dependent gated state update. Compared with the full $(A, B, C, D)$ selective-scan parameterization and low-rank discretization in the original Mamba, this formulation leads to a more regular computation path and is more suitable for hardware implementation [2].

With this notation, the simplified recurrence remains consistent with the general SSM description in Section 3.1: $h_t$ is the hidden state, while $x_t^{(s)}$ and $x_t^{(g)}$ are branch-wise features derived from the current input token. The main difference is that Slim-Mamba does not instantiate the full $(A, B, C, D)$ parameterization explicitly. Instead, the state-transition behavior is absorbed into the input-dependent coefficient $\lambda_t$ and the subsequent gated output path.

In parallel, the gate branch is passed through a SiLU activation, defined as

$$\mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}. \tag{3.13}$$

Therefore, the output gate is computed as

$$g_t = \mathrm{SiLU}(x_t^{(g)}). \tag{3.14}$$

The output gate then modulates the updated state through element-wise multiplication:

$$y_t = h_t \odot g_t. \tag{3.15}$$

This gating operation allows the model to regulate the effective information from the SSM output according to the current input. The result is then passed through a linear output projection and added back to the input tokens through a residual connection. The residual path helps preserve the original input information and improves training stability. Overall, Slim-Mamba retains the recurrent state update and gated modulation mechanisms of Mamba, while removing complex channel-fusion branches that provide limited benefit for the current task.

Compared with channel mamba, Slim-Mamba adopts a single-path backbone and reduces branch control and cross-channel fusion overhead. Compared with Vanilla Mamba v1, it simplifies the state-update parameterization and concentrates the main computation on regular operators such as linear projection, update gate generation, state update, element-wise multiplication, and output projection. This structure is more suitable for pipelined FPGA implementation and fixed-point quantization, and provides a clear datapath foundation for the subsequent Mamba-based indoor localization accelerator.

# Chapter 4
# Method

## 4.1 Hardware Architecture

This section presents the overall hardware architecture of the Slim-Mamba FPGA accelerator. The design is deployed on a Zynq MPSoC platform, where the processing system (PS) handles software control and DDR management, while the programmable logic (PL) implements the accelerator datapath. The PS-side DDR4 is used as off-chip runtime storage for input and output buffers. The input vector $x_t$ is read from DDR through the memory-mapped-to-stream (MM2S) channel of the AXI DMA and sent to the PL as an AXI-Stream. The output $y_t$ is written back to DDR through the stream-to-memory-mapped (S2MM) channel. The PS configures the accelerator through AXI-Lite control/status registers, including input preload, computation start, status polling, and interrupt control.

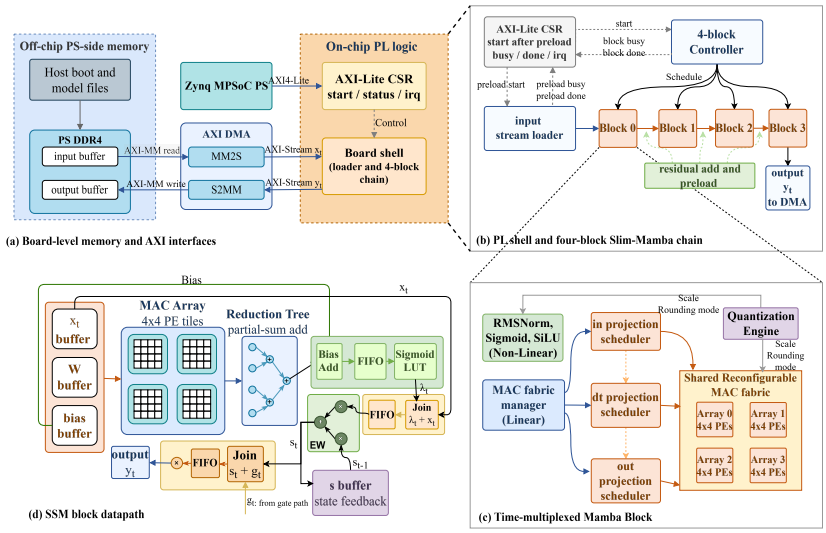



**Figure 4.1:** Overall hardware architecture of the Slim-Mamba accelerator.

Inside the PL, the board shell connects the external AXI interfaces to the compute pipeline formed by four Slim-Mamba blocks. The AXI-Lite CSR first starts the input stream loader, which writes the incoming AXI-Stream data into the on-chip input buffer of the first block. After the preload stage is completed, the shell allows the four-block controller to start computation. The controller schedules the four blocks sequentially. Between adjacent blocks, the output of the current block is added to the residual using saturated fixed-point arithmetic and then preloaded into the input buffer of the next block. The final block output is not passed through another block-level residual preload stage, but is directly streamed to the DMA as $y_t$.

Within each Slim-Mamba block, the main MAC-intensive stages share a reconfigurable MAC fabric. The input projection scheduler, the $\Delta$ projection scheduler, and the output projection scheduler generate tiled operands from on-chip weight and activation buffers. The MAC fabric manager selects the active scheduler and sends its operands to the shared $4 \times 4 \times 4$ MAC fabric. This fabric consists of four $4 \times 4$ PE arrays. By time-multiplexing the same fabric across different projection stages, the design avoids instantiating separate MAC arrays and reduces DSP and on-chip interconnect cost.

The design also uses a unified reconfigurable quantization stage. After MAC or element-wise computation, intermediate results are scaled, rounded, and saturated according to the current scheduler and datapath mode, and are then mapped back to the target fixed-point format. Therefore, the same quantization module can support different scale parameters across projection stages and element-wise operations. In the current implementation, weights, scale parameters, and nonlinear lookup tables are initialized into on-chip PL memories through memory initialization files, while DDR is mainly used for runtime input and output data movement.

## 4.2 Linear Layers

As discussed in Section 3.2, the Slim-Mamba algorithm can be mapped to three major hardware primitives. The first primitive is matrix-vector multiplication (MVM) implemented with multiply-accumulate (MAC) operations, which is mainly used for the input projection, the update-gate projection, and the output projection. The second primitive is element-wise multiplication (EWM), which is used for the weighted terms in the state update and for output gating. The third primitive is element-wise addition (EWA), which is used for the state update and residual addition. In the state-update datapath, these three primitives correspond to projection computation, recurrent state update, and output gating, respectively.

### 4.2.1 Reuse vs. Latency Trade-off

From a hardware design perspective, these primitives can be mapped with different resource-reuse strategies. One option is to build a unified reconfigurable PE array, where the processing elements switch among MAC, EWM, and EWA modes through a common mode signal. This approach maximizes the reuse of

multipliers, adders, and local buffers, thereby reducing area and resource cost. However, since different computation stages share the same array, this design may introduce mode-switching overhead, scheduling stalls, and a coarser pipeline granularity. Another option is to map the projection-heavy computations to a shared MAC fabric, while using lightweight vector EWM/EWA modules for state update and output gating. This design introduces some additional resources, but it allows projection, state update, and output gating to be connected in a finer-grained streaming pipeline.

Therefore, the current implementation adopts a compromise between resource reuse and pipeline granularity. The underlying PE array is capable of supporting MAC, EWM, and EWA modes through a unified mode signal. In the current Slim-Mamba accelerator, however, the input projection, update-gate projection, and output projection paths mainly use the shared MAC fabric in MAC mode. The element-wise multiply/add operations in the SSM state update and the element-wise multiplication in output gating are implemented using dedicated vector modules. In other words, the design preserves the reconfigurable capability of MAC/EWM/EWA execution, but functionally separates projection computation from element-wise computation to achieve better pipeline granularity and module decoupling.

This choice is also reflected in the fine-grained scheduling of the SSM datapath. In a coarse-grained design, different stages tend to execute sequentially, which improves resource reuse but increases end-to-end waiting time. In contrast, fine-grained streaming allows each MAC tile to be forwarded immediately to the following state update and output gating stages. In our throughput analysis with $N = 64$ tiles, the fine-grained schedule reduces latency from

$$T_A = 29N \tag{4.1}$$

to

$$T_B = 22N + 7, \tag{4.2}$$

that is, from 1856 cycles to 1415 cycles, corresponding to a 23.8% reduction. This schedule adds only about 12 DSPs per SSM compute path, which is acceptable compared with the achieved latency reduction. Based on this trade-off, the design adopts a fine-grained FIFO-based AXI-stream control scheme instead of relying entirely on a single reconfigurable array for all computations.

### 4.2.2 MAC Array Architecture Selection

The core workload of the linear layers can be abstracted as matrix-vector multiplication. Taking the dt projection as an example, its linear part can be expressed as

$$a_t = W_\lambda u_t + b_\lambda, \tag{4.3}$$

where $u_t$ is the state input, and $W_\lambda$ and $b_\lambda$ are the projection weight and bias, respectively. This computation is performed by the MAC fabric. Similarly, the input projection generates the state input and gate input from the input feature, while the output projection maps the gated result back to the block output dimension. These projections are all MAC operations and can therefore reuse the same MAC fabric with different input buffers, weight buffers, and output destinations.

Taking the state-update projection workload as an example, a typical linear layer can be modeled as the multiplication between a $256 \times 256$ matrix and a $256 \times 1$ vector, resulting in

$$256 \times 256 = 65,536 \tag{4.4}$$

MAC operations. Given a fixed number of processing elements, the ideal lower bound of the compute cycles can be estimated by

$$CC_{\text{min}} = \left\lceil \frac{\text{Total MAC operations}}{\#\text{PEs}} \right\rceil. \tag{4.5}$$

This formula only gives a theoretical lower bound. The actual latency is also affected by pipeline fill and drain, reduction, scheduling, and data delivery. Based on this workload, three candidate MAC array organizations are compared in Table 4.1. Here, the number of "+" symbols qualitatively indicates the relative wiring complexity.

A large $16 \times 16$ systolic array provides high spatial parallelism and a low theoretical cycle count. A systolic array is a regular grid of processing elements in which data are rhythmically passed between neighboring PEs, allowing multiply-accumulate operations to be performed in a pipelined dataflow, as shown in Fig. 4.2. However, for the projection workload in this design, the computation is dominated by GEMV (general matrix-vector multiplication) rather than large-batch GEMM (general matrix-matrix multiplication). Therefore, the two-dimensional data reuse advantage of a large systolic array is not fully exploited. The array effectively behaves like multiple narrow GEMV columns operating in parallel, while still introducing the routing, control, and data-supply pressure of a large two-dimensional PE array.

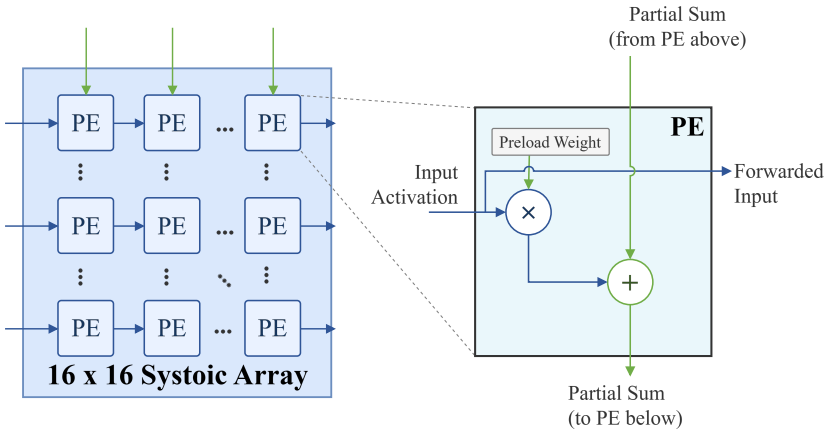



**Figure 4.2:** Example of a systolic array.

A $64 \times 1$ GEMV engine is more directly matched to matrix-vector multiplication. As illustrated in Fig. 4.3, the input vector is sent to a column of MAC units, where each PE multiplies one input element with the corresponding matrix weight;

the resulting products are then accumulated through a reduction tree to form the partial sum of one output channel. However, in a typical $64 \times 1$ GEMV engine, if all 64 MACs serve one output lane, 64 products are generated in one cycle and must be reduced into one partial sum for that output channel. This usually requires a 64-input adder tree, or a multi-stage pipelined adder tree. Although such a structure is suitable for pure GEMV computation, the large adder-tree fan-in may increase the number of adder stages, routing complexity, and the pressure of pipeline register insertion, making timing closure more challenging on FPGA.

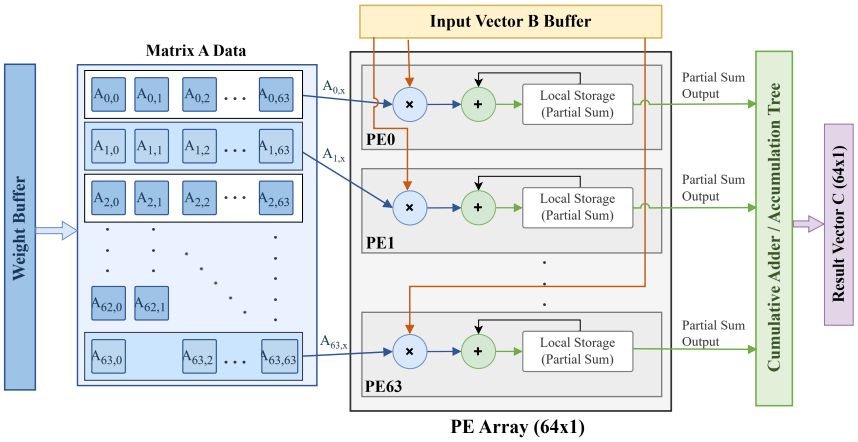



**Figure 4.3:** $64 \times 1$ GEMV engine.

The proposed $4 \times 4 \times 4$ MAC array organizes the 64 PEs as four cascaded $4 \times 4$ sub-arrays. Each sub-array processes a small input-weight tile, while the four sub-arrays are activated with a staggered schedule to combine local spatial parallelism and inter-array pipelining, as shown in Fig. 4.1 (d). The comparison above shows the advantage of this organization. Compared with a $64 \times 1$ GEMV engine, it provides higher vector-output parallelism and reduces the pressure of a large top-level reduction tree. Compared with a large $16 \times 16$ systolic array, it avoids excessive global routing and data-supply pressure for the MVM-dominated workload. Therefore, the $4 \times 4 \times 4$ structure is selected as a balanced shared MAC fabric in terms of resource cost, pipeline regularity, and timing closure.

**Table 4.1:** Comparison of candidate MAC array organizations for projection computation.

| | $16 \times 16$ Systolic | $64 \times 1$ GEMV | $4 \times 4 \times 4$ Pipeline |
|---|---|---|---|
| DSPs | 256 | 64 | 64 |
| BW (weights/cycle) | 256 | 64 | 64 |
| Clock cycles for $(256, 256)$ MVM | 256 | 1024 | 1024 |

At each cycle, a sub-array receives a 4-element slice of the input vector together with a $4 \times 4$ weight block. The first sub-array starts a new output tile, and the following sub-arrays join the same tile with a one-cycle offset. Weights

**Table 4.4:** Weight input scheduling for the pipelined MAC arrays.

| Cycle | Array1 Columns | Array2 Columns | Array3 Columns | Array4 Columns | Description |
|---|---|---|---|---|---|
| 1 | col0–3 | – | – | – | ARRAY1 preloads first $4 \times 4$ block (tile1). |
| 2 | col16–19 | col4–7 | – | – | ARRAY2 begins tile1. |
| 3 | col32–35 | col20–23 | col8–11 | – | ARRAY3 begins tile1 (3-cycle stagger). |
| 4 | col48–51 | col36–39 | col24–27 | col12–15 | ARRAY4 joins; pipeline full. |
| 5–16 | continue +16 stride | same | same | same | Steady-state loading of tile1. |
| 17 | **col256–259 → tile2** | col244–247 | col232–235 | col220–223 | **ARRAY1 starts tile2 (row4–7).** |
| 18 | col272–275 | **col260–263 → tile2** | col248–251 | col236–239 | **ARRAY2 switches to tile2.** |
| 19 | col288–291 | col276–279 | **col264–267 → tile2** | col252–255 | **ARRAY3 switches to tile2.** |
| 20 | col304–307 | col292–295 | col280–283 | **col268–271 → tile2** | **ARRAY4 switches; tile1 finishes.** |
| 21–37 | continue +16 stride | same | same | same | Steady-state operation for tile2. |
| 38 | **tile3 preload** | – | – | – | ARRAY1 starts tile3. |
| 39 | – | **tile3 preload** | – | – | ARRAY2 starts tile3. |
| 40 | – | – | **tile3 preload** | – | ARRAY3 starts tile3. |
| 41 | – | – | – | **tile3 preload** | ARRAY4 starts tile3 (3-cycle stagger). |
| 42–58 | continue +16 stride | same | same | same | Steady tile3 operation. |

are scheduled in 4-column blocks. For each sub-array, the column start index advances by a fixed stride of 16 every cycle, as summarized in Table 4.2. Within the same cycle, the four sub-arrays access distinct column blocks with fixed offsets $\{0, 4, 8, 12\}$, resulting in a constant 12-column spacing between adjacent active arrays, as shown in Table 4.3.

**Table 4.2:** Column starts per sub-array.

| Array | Column Start Points | Δ |
|---|---|---|
| Array1 | $0 \rightarrow 16 \rightarrow 32 \rightarrow 48 \rightarrow 64$ | +16 |
| Array2 | $4 \rightarrow 20 \rightarrow 36 \rightarrow 52$ | +16 |
| Array3 | $8 \rightarrow 24 \rightarrow 40$ | +16 |
| Array4 | $12 \rightarrow 28$ | +16 |

**Table 4.3:** 12-column array spacing.

| Cycle | A1→A2 Δ | A2→A3 Δ | A3→A4 Δ |
|---|---|---|---|
| 2 | 16–4 = **12** | – | – |
| 3 | 32–20 = **12** | 20–8 = **12** | – |
| 4 | 48–36 = **12** | 36–24 = **12** | 24–12 = **12** |
| 5 | 64–52 = **12** | 52–40 = **12** | 40–28 = **12** |

Also, the input vector $x_t$ follows the same staggered pipeline as the weight tiles. Each sub-array receives the activation slice that matches its current $4 \times 4$ weight block, and these slices are delayed together with the corresponding weights to keep the four sub-arrays aligned. After the 16 column groups of the current output-row tile have been consumed, the scheduler switches to the next output-row tile. Table 4.4 illustrates this tile-switching pattern.

### 4.2.3 Hardware Implementation of Linear Layers

An example of the array hardware implementation is shown in Fig. 4.1(d). Local accumulation is first performed inside the $4 \times 4$ MAC arrays, and the partial results generated by the four sub-arrays are then sent to a 4-input reduction tree to form the vector output. The reduced vector does not need to wait for all later partial vectors of the layer to be completed. Instead, it is forwarded through FIFO interfaces to the following stage, such as the bias-add or requantization stage in the SSM projection datapath. These FIFOs decouple the reduction output from the downstream pipeline and keep the data transfer stable under valid/ready control. Therefore, the $4 \times 4 \times 4$ structure uses small-granularity tiles, array cascade, cross-cycle accumulation, and FIFO-based streaming to distribute computation and reduction pressure into a more regular and localized structure.

## 4.3 Non-linear Layers

### 4.3.1 Normalization

In the Slim-Mamba block as shown in Fig. 3.3, the normalization layer is placed before the input projection to regulate the dynamic range of the input tokens. Since the subsequent linear projection, recurrent state update, and gating operations are sensitive to the scale of intermediate activations, large variations across channels or time steps may lead to numerical instability after fixed-point quantization. Therefore, normalization is introduced to rescale the input features into a more stable numerical range before they enter the main datapath.

Following the computation structure of Mamba-family blocks, where RMS normalization is commonly used as part of the block-level computation path [10, 11], this work adopts RMSNorm as the normalization layer in Slim-Mamba. For an input feature vector $\mathbf{x} = [x_0, x_1, \ldots, x_{D-1}]$, the RMS value is computed as

$$\mathrm{RMS}(\mathbf{x}) = \sqrt{\frac{1}{D}\sum_{i=0}^{D-1} x_i^2 + \epsilon}, \tag{4.6}$$

where $D$ denotes the feature dimension and $\epsilon$ is a small positive constant used to avoid division by zero. The normalized output is then computed as

$$\hat{x}_i = \frac{x_i}{\mathrm{RMS}(\mathbf{x})}. \tag{4.7}$$

From a hardware perspective, RMSNorm can be decomposed into square accumulation, mean scaling, square-root computation, and division. The square accumulation is implemented using multipliers and an adder tree. The mean scaling corresponds to division by the feature dimension $D$; when $D$ is a power of two, this operation can be implemented by a right shift. In contrast, the square-root and division operations are relatively expensive in a fixed-point FPGA datapath. Sutikno et al. investigated FPGA implementations of non-restoring square-root algorithms and showed that such digit-recurrence methods can be hardware friendly [17]. Motivated by this, our work adopts a trial-quotient square-root procedure which is consistent with digit-recurrence or non-restoring square-root algorithms. This algorithm is attractive for FPGA implementation because it relies on a regular sequence of shift, comparison, subtraction, and control operations, without requiring a general floating-point square-root unit, a large lookup table, or a large number of multipliers.

The concrete square-root computation is shown in Fig. 4.4. The input is the fixed-point value `mean_sq_q16` $+ \epsilon$, and both the partial remainder and the root register are initialized to zero. During each iteration, the next input bits are shifted into the partial remainder, and a trial value is generated from the current root estimate. If the partial remainder is greater than or equal to the trial value, the trial subtraction is accepted and the current root bit is set to one. Otherwise, the remainder is kept unchanged and the current root bit is set to zero. After all root bits have been processed, the module outputs `rms_q88`, which is the fixed-point RMS value used by the subsequent normalization stage.

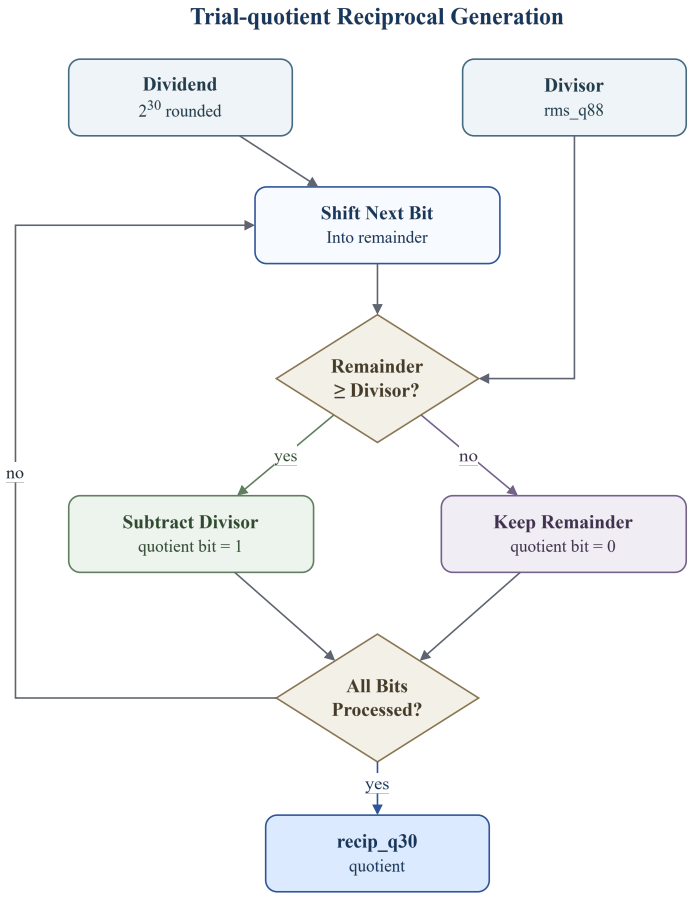



**Figure 4.4:** Trial-quotient square-root computation for the RMS value.

After `rms_q88` is obtained, the normalization still requires division by the RMS value. Directly instantiating a divider for each channel would lead to a high resource cost, especially because the same RMS value is shared by all channels of the current token. Therefore, the division in Eq. (4.7) is transformed into reciprocal approximation followed by multiplication:

$$\frac{x_i}{\mathrm{RMS}(\mathbf{x})} \approx x_i \cdot \widehat{\frac{1}{\mathrm{RMS}(\mathbf{x})}}. \tag{4.8}$$

The reciprocal-generation procedure is shown in Fig. 4.5. The module uses the fixed-point constant $2^{30}$ as the dividend and `rms_q88` as the divisor. A quotient is generated bit by bit using a trial-quotient division process. At each step, the next dividend bit is shifted into the partial remainder. If the remainder is greater than or equal to the divisor, the divisor is subtracted and the current quotient bit is set to one; otherwise, the remainder is kept and the quotient bit is set to zero. After all quotient bits have been generated, the output `recip_q30` represents the

Q30 fixed-point approximation of the RMS reciprocal. The normalized output is then produced by multiplying the input feature with `recip_q30`, followed by fixed-point shifting, rounding, and saturation.

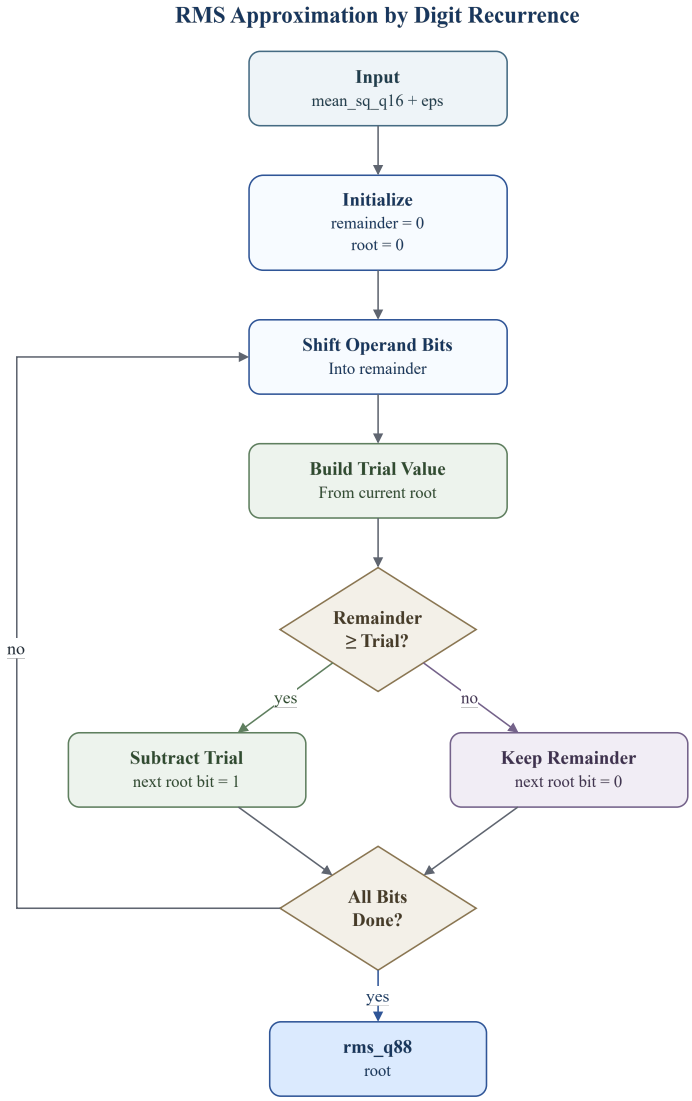



**Figure 4.5:** Reciprocal-based division approximation for RMSNorm.

A higher-precision Q30 reciprocal is used because the reciprocal acts as a shared scaling factor for all channels of the current token. If this reciprocal were represented with insufficient fractional precision, the same scaling error would be propagated to the entire normalized vector and further affect the following linear projection and state-update stages. Therefore, the reciprocal is computed with higher fractional precision and is only reduced to the target fixed-point format

after multiplication, rounding, and saturation. Experimental validation shows that this precision is sufficient to preserve the numerical accuracy of the fixed-point normalization unit and the final localization performance.

The reciprocal-multiplication formulation converts the normalization division into one reciprocal generation followed by multiple multiplications. The reciprocal can be computed once and then reused across all channels. Istoan and Pasca compared fixed-point FPGA implementations of reciprocal, square-root, and reciprocal-square-root functions, and pointed out that these functions often share common implementation structures, with the objective of optimizing the use of fixed FPGA resources such as block multipliers and block RAMs[18]. Therefore, replacing direct division with reciprocal approximation and multiplication provides a hardware-friendly implementation for the proposed fixed-point RMSNorm datapath.

Overall, the proposed normalization unit preserves the function of RMSNorm while avoiding costly general-purpose square-root and division units. The division by the feature dimension is implemented by shifting, the square root is computed using a trial-quotient digit-recurrence procedure, and the final division is implemented through reciprocal approximation and multiplication. As shown in Fig. 4.6, this hardware approximation remains close to the floating-point RMSNorm reference, with a maximum absolute error of 0.015625 and a mean absolute error of 0.004211 in the validation case.

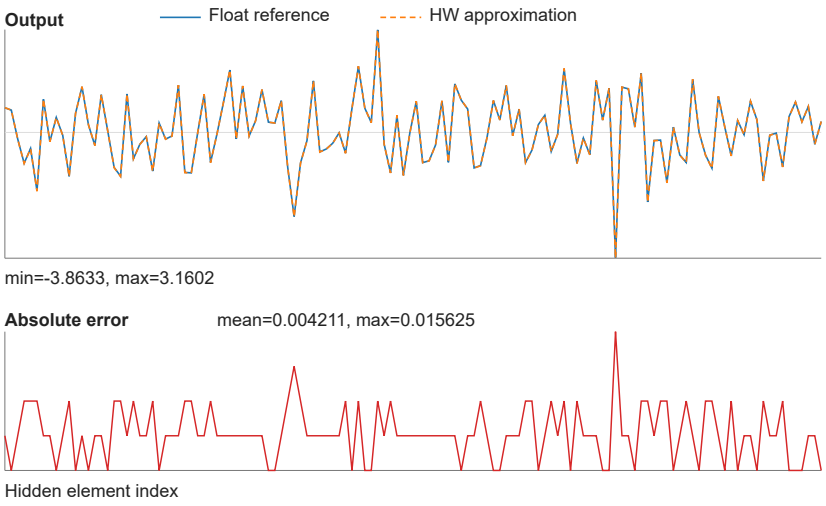



**Figure 4.6:** Comparison between floating-point RMSNorm and the fixed-point hardware approximation.

### 4.3.2 Sigmoid and SiLU

The sigmoid function is used in the SSM update path to generate bounded update coefficients. To realize the nonlinearity $\sigma(x) = 1/(1+e^{-x})$ with low FPGA overhead, this work implements a lookup-table (LUT) approximation. Before the lookup, the input is clamped to the interval $[-4, +4)$. This range is selected

from empirical workload statistics: sampled $dt\_proj$ values before the sigmoid are concentrated in this interval, so the LUT does not need to cover large saturation regions with fine resolution.

After clamping, the LUT address is generated by shifting the selected input domain to a non-negative index range:

$$\text{addr} = x_{\text{clamp}} + x_{\text{offset}}. \tag{4.9}$$

Here, $x_{\text{offset}}$ maps the lower bound of the clamped interval to address zero. A 2048-entry LUT is used to uniformly cover the effective sigmoid domain. Fig. 4.7 compares the floating-point sigmoid with the hardware LUT output, the error is below 0.02% showing that the chosen clamp range and table resolution provide a close approximation.

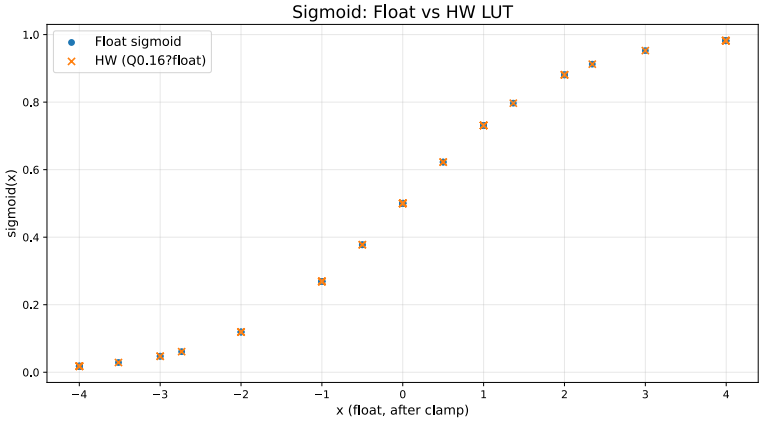



**Figure 4.7:** Comparison between floating-point sigmoid and the fixed-point LUT approximation.

The SiLU activation is implemented using the same sigmoid LUT path rather than a separate nonlinear unit. For an input $x$, SiLU is written as

$$\text{SiLU}(x) = x \cdot \sigma(x). \tag{4.10}$$

In hardware, the input vector is sent to the sigmoid LUT and, in parallel, delayed through a FIFO path. After the sigmoid value and the original input are aligned, an element-wise multiplier produces the SiLU output. This reuses the same clamp and LUT mechanism as the sigmoid unit, while adding only the alignment FIFO and vector multiplication required by the $x \cdot \sigma(x)$ formulation.

## 4.4 Quantization

Quantization converts floating-point values into fixed-point or integer representations so that the model can be hardware-friendly. In a floating-point model, values are stored with a flexible exponent and mantissa, while in a fixed-point

datapath the binary point is fixed and each value is represented by an integer together with a scale factor. In this work, a real-valued number $x$ is represented by an integer value $q$ and a scale factor $s$ and $\hat{x}$ is the reconstructed real value after dequantization:

$$x \approx \hat{x} = sq. \tag{4.11}$$

For a binary fixed-point format with $n$ fractional bits, the scale is $s = 2^{-n}$. In this thesis, the notation Q$m$.$n$ denotes a fixed-point format with $n$ fractional bits and a binary point placed $n$ bits from the least significant bit. For example, signed Q8.8 uses the scale $2^{-8}$, unsigned Q0.16 uses the scale $2^{-16}$, and signed Q1.15 uses the scale $2^{-15}$. The corresponding quantization and dequantization operations are written as

$$q = \mathrm{sat}_{[q_{\min}, q_{\max}]} \left( \mathrm{round} \left( \frac{x}{s} \right) \right), \tag{4.12}$$

$$\hat{x} = sq. \tag{4.13}$$

Here, round$(\cdot)$ denotes the selected rounding rule, and sat$(\cdot)$ clamps the integer result to the representable range.

### 4.4.1 Quantization Workflow

This subsection describes the quantization flow from the floating-point Slim-Mamba model to the fixed-point RTL implementation. In this design, quantization is not only a conversion of floating-point weights into integer values. The fixed-point rounding and saturation behavior in linear layers, nonlinear functions, and state updates must also be considered. Therefore, the proposed flow uses three validation levels: a Python model, a C++ fixed-point model, and an RTL implementation.

The Python model is used as the golden reference. It preserves the floating-point Slim-Mamba behavior and allows fast evaluation of different quantization settings on the final localization accuracy. The C++ fixed-point model then reproduces the main integer operations used in hardware, including multiplication, shifting, lookup-table access, rounding, and saturation. This level is closer to RTL than the Python model and is used to define the bit-true behavior of each operator. Finally, the RTL implementation receives the quantized weights, bias terms, scale factors, nonlinear LUTs, and control parameters. Its output is compared against the C++ fixed-point reference.

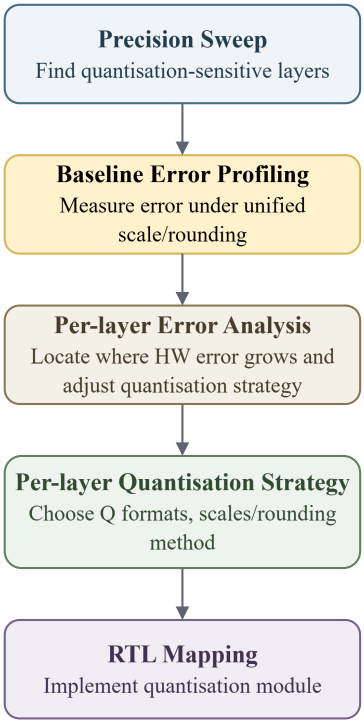



**Figure 4.8:** Quantization workflow from precision sweep to RTL mapping.

As shown in Fig. 4.8, the workflow starts with a precision sweep to determine the required precision for different computation stages. A baseline error profiling step is then performed using a unified requantization configuration. If the final output error is too large, per-layer error analysis is used to locate where the mismatch grows. Based on this analysis, the layer-wise quantization strategy is refined and then mapped to the RTL quantization modules and memory initialization files.

The purpose of this workflow is to keep the quantization decisions traceable across algorithm evaluation and hardware execution. Python reflects the floating-point behavior of the model, C++ fixes the bit-true integer semantics, and RTL verifies the final hardware behavior. This step-by-step process helps identify the main quantization error sources before FPGA implementation and keeps the deployed fixed-point accelerator consistent with the earlier model validation.

### 4.4.2 Error Analysis and Layer-wise Quantization Strategy

Before the bit-true RTL mapping, a fake-quantization precision sweep is first used to estimate the accuracy impact of different bit-width choices. In this step, quantize and dequantize operations are inserted in the software model to emulate low-precision effects, while the evaluation is still performed before RTL implementation. As shown in Table 4.5, the all-int16 configuration achieves a localization error close to the fake-quantization reference. The sweep also shows that the

`dt_proj`, `ssm_state`, and gate paths can be reduced to int8 with negligible loss, while `in_proj` and `out_proj` are more sensitive to precision reduction. However, the accelerator uses a shared reconfigurable MAC fabric for multiple projection stages. To keep the array control, datapath width, and on-chip memory organization simple, the final implementation adopts a unified 16-bit datapath instead of assigning different bit-widths to different stages.

**Table 4.5:** Fake-quantization precision sweep.

| Configuration | Precision override | Mean localization error (m) |
|---|---|---|
| Fake-quant reference | Software fake-quant baseline | 0.13695 |
| All-int16 | `ssm_state=int16`, `gate=int16` | 0.13520 |
| Mixed candidate | `dt_proj=int8`, `ssm_state=int8`, `gate=int8` | 0.13519 |
| `in_proj` int8 | `in_proj=int8`, `ssm_state=int16`, `gate=int16` | 0.76520 |
| `out_proj` int8 | `out_proj=int8`, `ssm_state=int16`, `gate=int16` | 0.24352 |

After the layer precision is selected, a unified fixed-scale quantization scheme is first evaluated as the baseline. In this baseline, the input projection, *dt* projection, output projection and selective-scan state use identity Q8.8 scales, while the MAC results are written back using round-to-nearest ties-to-even and saturation. Therefore, the real value represented by an integer $q$ is

$$x \approx \hat{x} = \frac{q}{2^8}. \tag{4.14}$$

Equivalently, the quantization scale is $s = 2^{-8} = 1/256$. When two Q8.8 values are multiplied, the product contains 16 fractional bits. After MAC accumulation, the result is shifted right by 8 bits to return to Q8.8:

$$z = \frac{\text{acc}}{2^8}, \tag{4.15}$$

where acc is the integer accumulation result before the final Q8.8 conversion.

The shifted value is rounded using round-to-nearest-even. This rule rounds a value to the closest integer; when the value lies exactly halfway between two integers, it chooses the even integer to reduce systematic rounding bias:

$$\text{RNE}(z) = \begin{cases} \lfloor z \rfloor, & z - \lfloor z \rfloor < 0.5, \\ \lceil z \rceil, & z - \lfloor z \rfloor > 0.5, \\ \text{the nearest even integer}, & z - \lfloor z \rfloor = 0.5. \end{cases} \tag{4.16}$$

Finally, the rounded integer is clamped to the signed 16-bit range:

$$\text{sat}_{16}(q) = \min(\max(q, -32768), 32767). \tag{4.17}$$

Since signed int16 ranges from $-32768$ to 32767, the corresponding real-value range of signed Q8.8 is

$$[-32768/256,\ 32767/256] = [-128,\ 127.996].$$

The nonlinear layers are also mapped with fixed Q formats. The sigmoid LUT is generated offline. Each table entry stores a pre-quantized unsigned Q0.16 approximation of the corresponding floating-point sigmoid value:

$$q_\sigma = \text{sat}_{[0,65535]} \left( \text{round}_{\text{nearest}} \left( \sigma(x) 2^{16} \right) \right), \tag{4.18}$$

where $q_\sigma$ is the stored LUT value and $\text{round}_{\text{nearest}}(\cdot)$ denotes the offline rounding to the nearest integer used during LUT generation. At runtime, the Q8.8 input is first clamped to the sigmoid range $[-4, +4)$, which is selected based on the observed value distribution in real data. After clamping, the input is shifted and converted to an unsigned integer index, and the LUT address is then generated using the fixed-point address mapping in Eq. (4.9).

The SiLU path reuses the same Q0.16 sigmoid output. The Q0.16 sigmoid value is multiplied by the signed Q8.8 input and then shifted back to Q8.8:

$$q_{\text{SiLU}} = (q_\sigma q_x) \gg 16, \tag{4.19}$$

where $q_x$ is the signed input integer Q8.8.

RMSNorm uses a Q30 reciprocal factor to preserve the precision of the shared normalization scale. The reciprocal integer is computed as

$$r_q = \text{round}_{\text{nearest}} \left( \frac{2^{30}}{\text{RMS}(\mathbf{x})} \right), \qquad \widehat{r} = \frac{r_q}{2^{30}}. \tag{4.20}$$

Here, $r_q$ is the stored Q30 reciprocal integer, $\widehat{r}$ is the dequantized approximation of $1/\text{RMS}(\mathbf{x})$, and $\text{round}_{\text{nearest}}(\cdot)$ denotes the rounding to the nearest integer during reciprocal generation. The Q30 format gives a reciprocal resolution of $2^{-30}$. A higher fractional precision is used here because the reciprocal is shared across all feature channels in one RMSNorm operation, so its quantization error is applied to the entire normalized vector.

After reciprocal multiplication, the Q30 product is shifted back to the Q8.8 output format. Before this shift, a signed round-to-nearest rule is applied:

$$q_{\text{norm}} = \begin{cases} \lfloor z \rfloor, & z - \lfloor z \rfloor < 0.5, \\ \lceil z \rceil, & z - \lfloor z \rfloor \geq 0.5. \end{cases} \tag{4.21}$$

where $z$ is the scaled value after the Q30 factor has been removed. The rounded result is then clamped to the signed Q8.8 range using the same saturation definition as in Eq. (4.17).

However, with the above baseline quantization settings, the output differs substantially from the floating-point output, with a mean absolute error (MAE) of 0.17987 and a mean localization error of 0.35543 m. To locate the error sources, the Slim-Mamba block is decomposed according to the data flow in Fig. 4.9, and the error is measured after each computation stage. The corresponding MAE values are listed in Table 4.6. This stage-level analysis shows that the largest error increase occurs in the selective scan stage. Since this stage repeatedly updates the recurrent state, its value range changes across channels and time steps, so a single fixed Q8.8 scale is not sufficient to represent both small and large state values. To reduce this error, the selective-scan state is changed to a dynamic-scale representation, where the scale is derived from the channel range. This reduces the selective-scan MAE from 0.127090 to 0.085484 and also improves the following gating, output projection, and block-output errors.

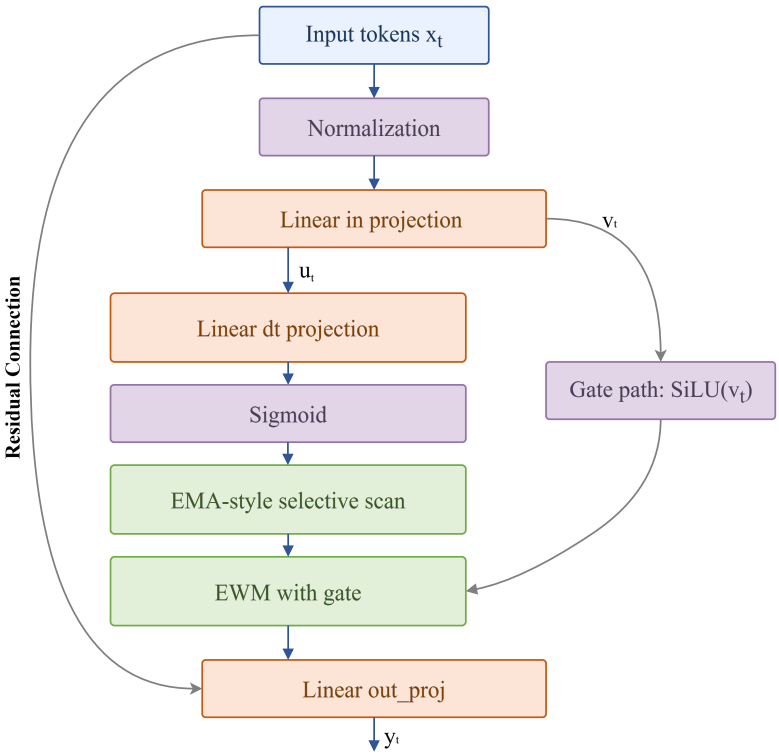



**Figure 4.9:** Slim-Mamba block flow used for stage-level quantization error analysis.

**Table 4.6:** Stage-level quantization error before and after dynamic scaling.

| Stage | Fixed-scale MAE | Dynamic-scale MAE |
|---|---|---|
| `norm` | 0.004498 | 0.004498 |
| `inproj_u` | 0.010917 | 0.010917 |
| `inproj_z` | 0.011069 | 0.011069 |
| `sigmoid_u` | 0.006183 | 0.006183 |
| `silu_gate` | 0.005931 | 0.005931 |
| `dtproj` | 0.010139 | 0.010139 |
| `dt_sigmoid` | 0.002118 | 0.002118 |
| `selective_scan` | 0.127090 | 0.085484 |
| `ewm_gating` | 0.033001 | 0.029827 |
| `outproj` | 0.032225 | 0.031243 |

The improvement is also reflected in the final end-to-end evaluation in Table 4.7. The early fixed-scale baseline has a large output mismatch and a much higher localization error. After the dynamic-scale state representation is introduced, the hardware output MAE is reduced to 0.02081, and the final mean local-

ization error becomes 0.14103 m, which is close to the fake-quantization reference of 0.13695 m.

**Table 4.7:** End-to-end quantization evaluation.

| Evaluation | Samples | HW vs. float MAE | Mean localization error (m) |
|---|---|---|---|
| fixed-scale | 32 | 0.17987 | 0.35543 |
| dynamic-scale | 34,505 | 0.02081 | 0.14103 |
| Fake-quant reference | 34,505 | – | 0.13695 |

Based on the above analysis, the final layer-wise quantization strategy is summarized in Table 4.8. Most intermediate values use signed Q8.8 with a scale of 1/256. The sigmoid-related values use unsigned Q0.16, which matches the range of the sigmoid output. The selective-scan coefficient $\lambda$ is represented as signed Q1.15, while the selective-scan state stored in SRAM uses a dynamic signed int16 scale. The RMSNorm reciprocal uses a higher-precision Q30 format to preserve the accuracy.

For rounding, round-to-nearest-even (RNE) is used in the projection requantization path and in the selective-scan state conversion path. These paths include repeated MAC accumulation, state update, and scale multiplication, where biased rounding errors can accumulate across channels and layers. The sigmoid LUT is rounded offline using nearest rounding (NR) when the table is generated, while the runtime sigmoid path only performs clamping and table lookup. RMSNorm also uses a signed NR conversion after reciprocal multiplication. The value denoted as `dt_lambda` is the Q0.16 sigmoid output generated after the $dt$ projection. It is then converted to the the Q1.15 selective-scan coefficient $\lambda$ by a one-bit right shift before the selective-scan update because the state update needs both $\lambda$ and $1-\lambda$. In Q1.15, the value 1.0 is represented as $2^{15}$, so $1-\lambda$ can be computed directly as $2^{15}-q_\lambda$. This step is a value-format conversion between two fixed-point representations, rather than a separate quantization or rounding operation.

For saturation, each format is clamped to its valid representable range. Signed Q8.8 values are clamped to the signed Q8.8 range $[-32768, 32767]$. Unsigned Q0.16 sigmoid-related values are clamped to $[0, 65535]$, while the selective-scan coefficient is limited to the sigmoid range $[0, 1]$ in Q1.15. The dynamic-scale state is also stored as a signed int16 value under its selected runtime scale. These saturation rules prevent overflow or sign errors after multiplication, accumulation, state update, and residual addition.

**Table 4.8:** Final layer-wise quantization strategy.

| Stage | Scale | Q format | Rounding | Saturation |
|---|---|---|---|---|
| RMSNorm output | 1/256 | Q8.8 | RNE | $[-32768, 32767]$ |
| RMSNorm reciprocal | $1/2^{30}$ | Q30 | NR | $[-32768, 32767]$ |
| `in_proj` | 1/256 | Q8.8 | RNE | $[-32768, 32767]$ |
| `dt_proj` | 1/256 | Q8.8 | RNE | $[-32768, 32767]$ |
| `out_proj` | 1/256 | Q8.8 | RNE | $[-32768, 32767]$ |
| `gate_y` | 1/256 | Q8.8 | RNE | $[-32768, 32767]$ |
| Sigmoid LUT output | 1/65536 | Q0.16 | NR | $[0, 65535]$ |
| SiLU sigmoid path | 1/65536 | Q0.16 | NR | $[0, 65535]$ |
| `dt_lambda` | 1/65536 | Q0.16 | NR | $[0, 65535]$ |
| Selective-scan $\lambda$ | 1/32768 | Q1.15 | No rounding, Shift | $[0, 1]$ |
| Selective-scan state | Dynamic | Q8.8 | RNE | $[-32768, 32767]$ |

### 4.4.3 Quantization Hardware Module Design

Instead of implementing a separate quantization unit for each layer, this work uses a shared configurable quantization engine. As shown in Fig. 4.10, the engine receives an input value, a selected scale, and several control modes. The input is first extended according to the input sign mode. It is then multiplied by either a fixed scale or a runtime dynamic scale. The scaled product is shifted to the target binary point, rounded according to the rounding mode, and finally converted to the target output range. The saturation mode selects whether the final value is clamped to the legal range or wrapped to the target bit width. This common structure is used after MAC accumulation, Q-format conversion, state update, scale multiplication, and residual addition. Compared with layer-specific quantization logic, the shared engine reduces duplicated RTL, keeps the rounding and saturation behavior consistent across the accelerator, and allows the scale, rounding, and saturation policies found in the software analysis to be mapped directly to hardware configuration parameters.

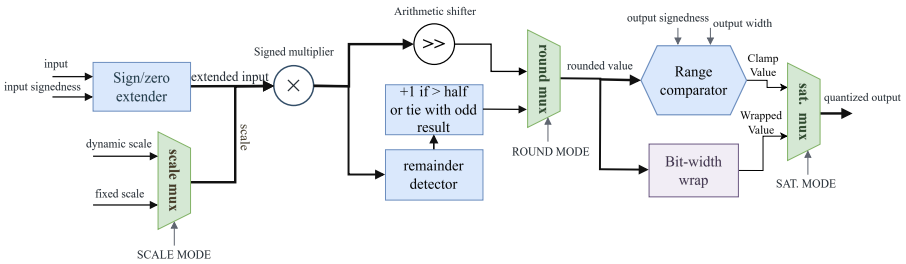



**Figure 4.10:** Configurable fixed-point quantization engine for one vector lane.

Dynamic scaling is used in the selective-scan state path, where the recurrent state is stored in SRAM and updated over time. The state range varies across channels and tokens, so a single fixed Q8.8 scale can either saturate large values or lose precision for small values. The runtime scale generator therefore derives scale

factors from the current state input range and produces the scale values required for state storage and conversion back to Q8.8. Since the selective-scan datapath is stream based, the token, state address, update coefficient, and corresponding runtime scales must remain aligned. For this purpose, the design uses the ping-pong buffer structure shown in Fig. 4.11. While one bank receives the incoming token and its runtime scales, the other bank can provide an already aligned token-scale pair to the downstream state update unit. The write and read bank selectors alternate between the two banks, which hides the scale-generation latency and preserves the ordering of the streaming data. This mechanism allows dynamic scaling to be inserted into the state update path without breaking the fine-grained pipeline.

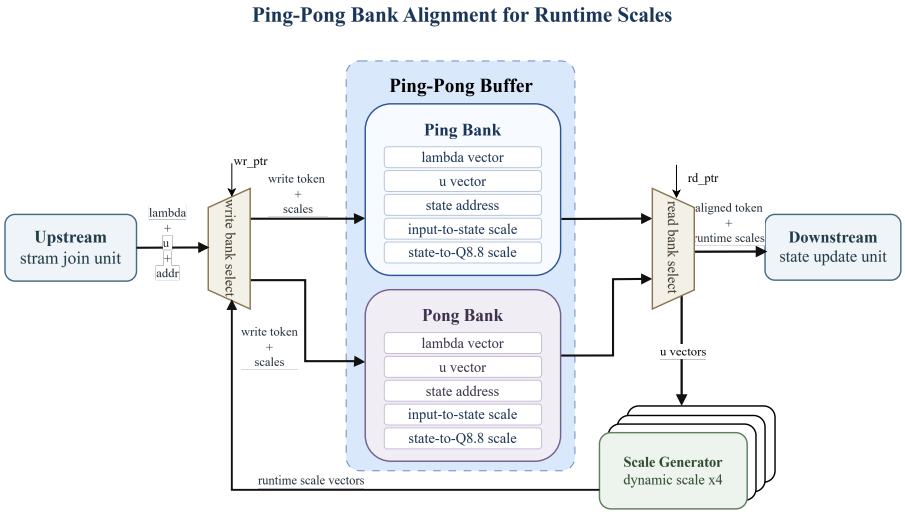



**Figure 4.11:** Ping-pong buffer alignment for runtime dynamic scales in the selective-scan path.

# Chapter 5
# Results and Discussion

## 5.1 Software Model Evaluation

This section evaluates the software models before the FPGA-specific results are discussed. Table 3.3 shows that channel mamba achieves the lowest mean localization error among the compared software models. However, that best result uses an augmented input that includes 100 additional channel-amplitude features and is therefore not a strictly input-matched comparison against Slim-Mamba. To make this distinction explicit, Fig. 5.1 compares the final Slim-Mamba model with two saved channel mamba reference runs: one using the same CIR+Power input as Slim-Mamba and one using the augmented CIR+Power plus amplitude input. The augmented-input channel mamba run yields the most favorable error distribution, whereas the input-matched channel mamba run is consistently worse than Slim-Mamba over most of the plotted range. These results indicate that channel mamba benefits from the additional multi-channel amplitude cues, but that advantage is not retained under the same input condition used by Slim-Mamba. For the subsequent hardware study, Slim-Mamba remains the more suitable deployment-oriented model because it combines competitive accuracy with much smaller model size, shorter training time, and a simpler datapath.

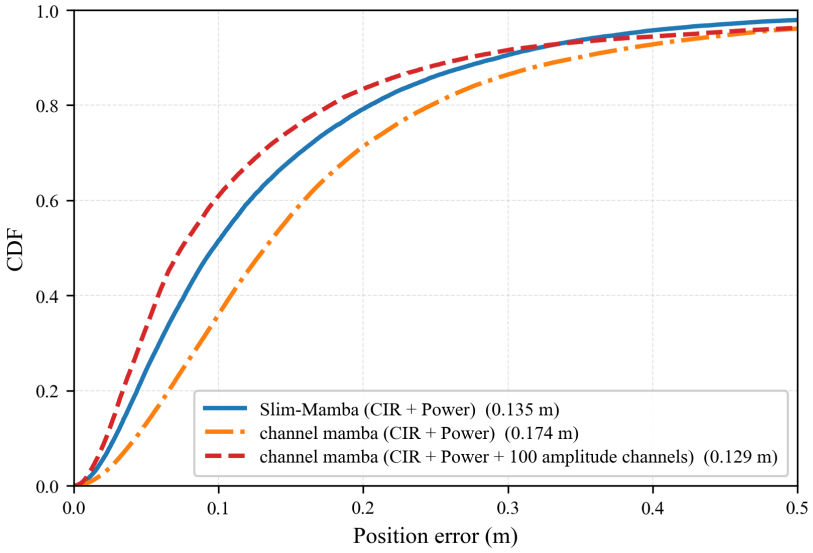



**Figure 5.1:** Cumulative distribution of localization error for Slim-Mamba and two channel mamba reference runs under matched and augmented input settings.

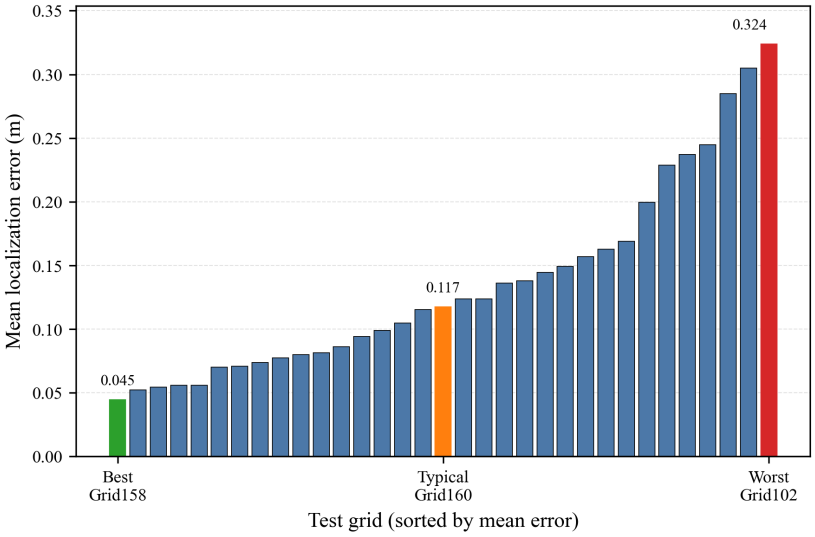



**Figure 5.2:** Per-grid mean localization error of the final Slim-Mamba model on the test set, sorted from best to worst.

The grid-level ranking in Fig. 5.2 shows that the final Slim-Mamba model maintains a stable error level over most of the test set, while the remaining variation

across grids is correlated with the local trajectory coverage around each query position. When a test trajectory lies in a region that is surrounded by more training trajectories, the localization error is generally lower; when the nearby trajectory coverage becomes sparser, the error tends to increase. Therefore, Fig. 5.3 presents representative examples from the low-error, mid-error, and high-error parts of the ranking to illustrate this spatial trend on the test set.

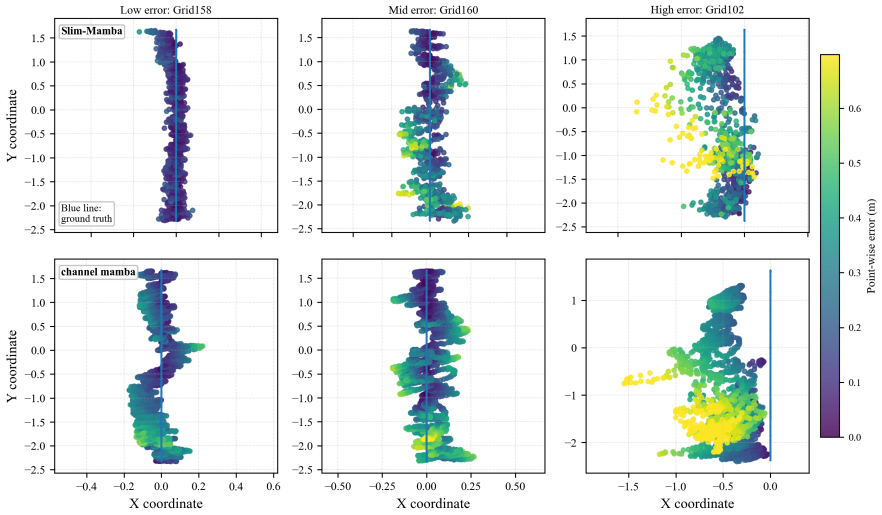



**Figure 5.3:** Trajectory-level comparisons on low-, mid-, and high-error grids for Slim-Mamba (top) and channel mamba (bottom).

After selecting Slim-Mamba as the deployment-oriented model, the temporal window length $K$ and patch setting are ablated. Table 5.1 shows that $K = 1$ and $K = 2$ both achieve the best test mean error of about 0.135 m, while longer temporal windows gradually increase the test error. The CDF comparison in Fig. 5.4 shows the same trend from a distributional perspective: short-window configurations keep more samples in the low-error region, whereas larger $K$ values mainly broaden the error tail. This suggests that, under the current LuViRA sampling density, extending the temporal context introduces more redundancy than useful additional localization cues. Therefore, short-window Slim-Mamba is used as the software model target for the subsequent hardware implementation.

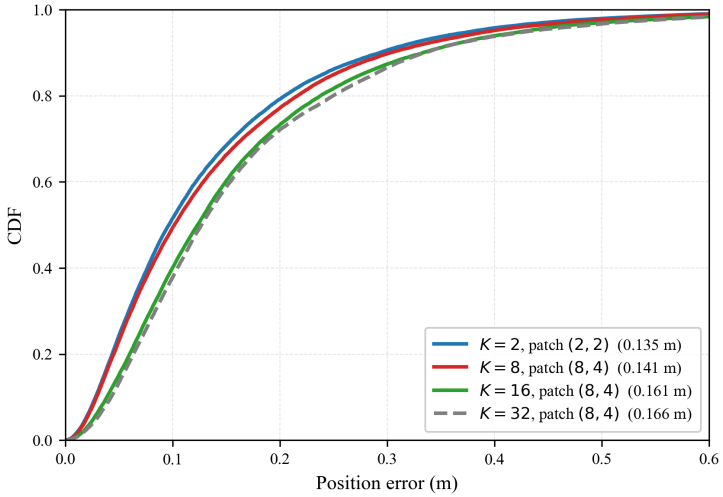



**Figure 5.4:** Cumulative distribution of localization error for representative Slim-Mamba temporal-window configurations.

**Table 5.1:** Ablation on window length $K$ with patch length and stride variants.

| $K$ (patch length, stride) | Eval mean err. (m) | Test mean err. (m) |
|---|---|---|
| 1 (1,1) | 0.10068 | **0.135** |
| 2 (2,2) | 0.09557 | **0.135** |
| 4 (2,2) | 0.09814 | 0.139 |
| 8 (2,2) | 0.09405 | 0.141 |
| 4 (4,4) | 0.09983 | 0.137 |
| 6 (4,4) | 0.09839 | 0.138 |
| 8 (4,4) | 0.09749 | 0.144 |
| 8 (8,4) | 0.09744 | 0.152 |
| 12 (8,4) | 0.09286 | 0.156 |
| 16 (8,4) | 0.09225 | 0.161 |
| 24 (8,4) | 0.09432 | 0.159 |
| 32 (8,4) | 0.09328 | 0.166 |

## 5.2 Accuracy

This section evaluates the localization accuracy across three stages: the floating-point Slim-Mamba model, the fake-quantized model, and the hardware fixed-point model. The floating-point model is used as the software reference. The fake-quantized model inserts quantize and dequantize operations during software inference, while all intermediate tensors are still accumulated in floating point, and is used to evaluate the effect of the selected bit-widths without introducing RTL-

specific arithmetic. The hardware fixed-point model further follows the integer datapath used in RTL, including fixed-point multiplication, shifting, rounding, saturation, lookup-table nonlinear functions, and dynamic scaling in the selective-scan state path. This layered comparison separates the model accuracy, software quantization error, and hardware fixed-point mapping error.

**Table 5.2:** Accuracy comparison across floating-point, fake-quantized, and hardware fixed-point models.

| Model stage | Numerical format | Output MAE vs. float | Mean localization error (m) |
|---|---|---|---|
| Floating-point Slim-Mamba | Float32 | – | 0.13500 |
| Fake-quantized Slim-Mamba | Software fake quantization | – | 0.13695 |
| Hardware fixed-point model | Fixed-point with dynamic scale | 0.02081 | 0.14103 |

Table 5.2 shows that software fake quantization introduces almost no additional localization error compared with the floating-point Slim-Mamba model. This indicates that the unified 16-bit datapath and the selected Q formats are sufficient for the main computation paths. The hardware fixed-point model increases the mean localization error from 0.13500 m to 0.14103 m. It also gives an output MAE of 0.02081 with respect to the floating-point output. This confirms that fixed-point arithmetic does introduce numerical differences, but these differences are not strongly amplified in the final localization result.

The main accuracy loss therefore comes from the hardware fixed-point mapping stage rather than from the software fake-quantization stage. This is expected because the hardware fixed-point model includes concrete RTL-oriented numerical behavior, such as round-to-nearest-even, saturation, LUT-based nonlinear approximation, fixed-point scale multiplication, and dynamic scaling for the selective-scan state. Nevertheless, the final mean localization error remains below 0.15 m and is only about 0.006 m higher than the floating-point result. This validates the quantization strategy described in Section 4.4.2, including the unified 16-bit datapath, Q8.8/Q0.16/Q1.15 formats, saturation handling, and selective-scan dynamic scaling.

## 5.3 Performance

This section evaluates the FPGA accelerator in terms of throughput, resource utilization, power, and block-level latency. Here, one frame denotes one end-to-end input window processed by the accelerator. The target real-time requirement is 100 frames/s. The implemented design closes timing at 100 MHz and achieves an estimated throughput of 471 frames/s, which is about 4.7× higher than the requirement. This corresponds to an average processing time of about 2.12 ms per frame, compared with the 10 ms frame interval allowed by the target requirement.

**Table 5.3:** Throughput summary at 100 MHz.

| Metric | Value |
|---|---|
| Target clock frequency | 100 MHz |
| Required throughput | 100 frames/s |
| Achieved throughput | 471 frames/s |
| Average time per frame | 2.12 ms |
| Requirement time per frame | 10 ms |

Table 5.4 reports the post-synthesis resource utilization under the 100 MHz constraint. The design uses 22.21% of LUTs, 14.11% of flip-flops, 30.59% of BRAM tiles, and 19.68% of DSP blocks. The BRAM usage mainly comes from on-chip weight memories, intermediate buffers, state SRAM, and lookup tables. The DSP usage is dominated by the shared MAC fabric, RMSNorm, element-wise multiplication, and scale-related fixed-point multiplications. Overall, the utilization remains well below the device capacity, leaving room for further optimization or model scaling.

**Table 5.4:** Post-synthesis resource utilization at 100 MHz.

| Resource | Used | Available | Utilization |
|---|---|---|---|
| LUT | 60868 | 274080 | 22.21% |
| FF | 77345 | 548160 | 14.11% |
| BRAM tiles | 279 | 912 | 30.59% |
| DSP | 496 | 2520 | 19.68% |
| Bonded IOB | 0 | 328 | 0% |

The on-chip power estimate is summarized in Table 5.5. The total on-chip power is 4.292 W, including 3.562 W dynamic power and 0.729 W static power. The processing system contributes 2.622 W of the dynamic power, which is about 74% of the dynamic component. This indicates that the total system power is determined not only by the programmable-logic compute array, but also by PS-side control, DDR access, and DMA-based data movement.

**Table 5.5:** On-chip power estimate.

| Power item | Power | Share |
|---|---|---|
| Total on-chip power | 4.292 W | 100% |
| Dynamic power | 3.562 W | 83% |
| Static power | 0.729 W | 17% |
| PS dynamic power | 2.622 W | 74% of dynamic power |

Before optimizing the hardware schedule, a function-level profiling study is used to identify the dominant computation. As shown in Table 5.6, the SSM path is the most important part of the workload. The SSM $dt$ projection accounts for 36.18% of the profiled time, and the selective-scan state update accounts for 41.34%. Together, they contribute more than 77% of the listed execution time.

This motivates the fine-grained pipeline design for the SSM path, since improving this part has the largest impact on the end-to-end block latency.

**Table 5.6:** Function-level profiling summary.

| Function | Time | Share (%) |
|---|---|---|
| Linear projection (in/out) | 30 | 7.75 |
| Normalization (RMSNorm) | 57 | 14.73 |
| SSM *dt* projection (MVM + sigmoid) | 140 | 36.18 |
| SSM selective scan (state update) | 160 | 41.34 |
| Total (listed) | 387 | 100.00 |

Table 5.7 gives the per-block latency breakdown. The total latency of one Slim-Mamba block is 5301 cycles, or 53.01 $\mu$s at 100 MHz. Input projection takes 2305 cycles, while the complete SSM part takes 1828 cycles. These two parts are useful for comparison because their main projection workloads have the same theoretical number of MAC operations. In the RTL configuration, the input projection maps a 128-dimensional vector to a 512-dimensional output, corresponding to $128 \times 512 = 65{,}536$ MAC operations. The SSM *dt* projection maps a 256-dimensional vector to a 256-dimensional vector, also corresponding to $256 \times 256 = 65{,}536$ MAC operations. However, the SSM path also includes sigmoid generation, selective-scan state update, element-wise operations, and scale alignment. Even with these extra operations, the SSM latency remains lower than the input-projection latency. This is mainly due to the fine-grained FIFO-based pipeline: the *dt* projection output is forwarded to the selective-scan and gating-related stages at tile granularity, instead of waiting for the full vector to be generated.

**Table 5.7:** Latency breakdown per Slim-Mamba block.

| Category | Clock cycles | Share |
|---|---|---|
| RMSNorm | 134 | 2.5% |
| Input projection | 2305 | 47.25% |
| Ping-pong buffer | 199 | 3.75% |
| SSM | 1828 | 34.48% |
| Output projection | 833 | 15.71% |
| Total | 5301 | 100% |

Overall, the accelerator satisfies the real-time throughput target with a large margin while keeping resource utilization moderate. The profiling and latency results also confirm the effectiveness of the SSM fine-grained pipeline: the most critical software workload is mapped to a streaming hardware schedule that reduces waiting time between the projection, nonlinear, and state-update stages.

## 5.4 FPGA On-board Validation

The final step is to validate that the FPGA platform can execute the complete hardware path on board. The purpose of this test is not to run the full LuViRA

test set, but to confirm that the processing system, DDR buffers, AXI DMA channels, AXI-Lite control registers, stream loader, four-block compute chain, and output write-back path are all connected correctly. The validation flow is shown in Fig. 5.5.

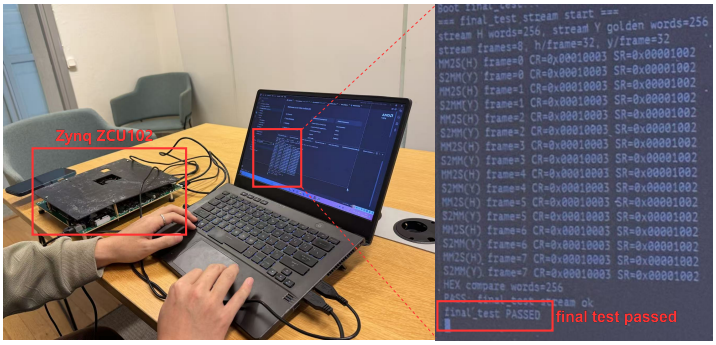



**Figure 5.5:** FPGA on-board validation flow.

In the current bring-up experiment, the SD card boot and model-file loading path is not used as part of the test. Instead, the host-side program prepares a large input matrix file containing a subset of validation samples. This matrix is loaded by the processing system into the DDR input buffer. The PS then programs the AXI DMA and the AXI-Lite control registers, starts the input preload and computation sequence, and waits for the completion status from the PL. During execution, the MM2S DMA channel streams the input matrix from DDR to the PL, and the board shell feeds the data into the four-block Slim-Mamba chain. After computation, the S2MM DMA channel writes the output stream back to the DDR output buffer, where it is read by the host-side program for checking.

The on-board test uses only a partial dataset, so it should be interpreted as a platform validation rather than a full accuracy evaluation. Nevertheless, the test passes with the expected completion status and output transfer behavior. This confirms that the main FPGA platform components are functional as an integrated system: PS-side control, DDR buffering, AXI-Lite register access, AXI DMA streaming, PL-side input loading, four-block computation, and output write-back are all exercised in one run. Therefore, the current result demonstrates that the accelerator has been successfully brought up on the FPGA platform.

# Chapter 6
# Conclusion

## 6.1 Conclusion

This thesis investigated the use of Mamba-family models for Massive MIMO indoor localization and developed a hardware-oriented Slim-Mamba accelerator on FPGA. The work combined software evaluation, fixed-point quantization, and RTL implementation. Based on LuViRA experiments, Slim-Mamba was selected as a deployment-oriented model because it keeps the recurrent update and gating behavior needed for localization while avoiding unnecessary multi-branch fusion and full selective-scan complexity.

The accelerator was implemented on a Zynq MPSoC platform with PS-side DDR/DMA control and a PL-side four-block Slim-Mamba compute chain. The design uses a shared $4 \times 4 \times 4$ MAC fabric, hardware nonlinear units, configurable quantization modules, and a dynamic-scale state path. The hardware fixed-point model reaches a mean localization error of 0.14103 m, close to the floating-point result of 0.13500 m. The RTL design closes timing at 100 MHz and achieves 471 frames/s, exceeding the 100 frames/s requirement, while reconfigurable MAC architecture keeps a low utilization (22.21% LUTs, 14.11% FFs, 30.59% BRAM tiles, and 19.68% DSPs). Board-level validation confirms that PS control, DDR buffering, AXI DMA streaming, PL computation, and output write-back are connected and functional.

## 6.2 Future Work

Although the current design demonstrates the feasibility of a Slim-Mamba FPGA accelerator, several directions remain for future work.

- **Full end-to-end board evaluation:** The current on-board validation uses a partial input matrix and does not yet use SD-card model loading. Future work should run the full LuViRA set on board and integrate model loading, DDR buffer preparation, and output checking.
- **Finer-grained whole-architecture pipeline:** In the current design, the SSM path is fine-grained, while RMSNorm, input projection, output projection, residual preload, and block-level scheduling still use coarser boundaries. These stages can be further overlapped to reduce end-to-end latency.

# References

[1] G. Tian, I. Yaman, M. Sandra, X. Cai, L. Liu, and F. Tufvesson, "Deep-learning-based high-precision localization with massive MIMO," *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 2, pp. 19–33, 2024.

[2] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023. [Online]. Available: https://arxiv.org/abs/2312.00752

[3] A. Bourdoux, A. N. Barreto, B. van Liempd, C. H. M. de Lima, D. Dardari, D. Belot, E. S. Lohan, G. Seco-Granados, H. Sarieddeen, H. Wymeersch *et al.*, "6G white paper on localization and sensing," 2020. [Online]. Available: https://arxiv.org/abs/2006.01779

[4] S. E. Trevlakis, A.-A. A. Boulogeorgos, D. Pliatsios, J. Querol, K. Ntontin, P. Sarigiannidis, S. Chatzinotas, and M. Di Renzo, "Localization as a key enabler of 6G wireless systems: A comprehensive survey and an outlook," *IEEE Open Journal of the Communications Society*, vol. 4, pp. 2733–2801, 2023.

[5] F. Mogyorósi, P. Revisnyei, A. Pašić, Z. Papp, G. Tóth, and P. Varga, "Positioning in 5G and 6G networks—a survey," *Sensors*, vol. 22, no. 13, p. 4757, 2022.

[6] A. S. Yaro, F. Maly, and P. Prazak, "A survey of the performance-limiting factors of a 2-dimensional RSS fingerprinting-based indoor wireless localization system," *Sensors*, vol. 23, no. 5, p. 2545, 2023.

[7] G. Tian, I. Yaman, M. Sandra, X. Cai, L. Liu, and F. Tufvesson, "High-precision machine-learning based indoor localization with massive MIMO system," in *ICC 2023 – IEEE International Conference on Communications*, Rome, Italy, 2023, pp. 3690–3695.

[8] A. Colpaert, S. De Bast, R. Beerten, A. P. Guevara, Z. Cui, and S. Pollin, "Massive MIMO channel measurement data set for localization and communication," *IEEE Communications Magazine*, vol. 61, no. 9, pp. 114–120, Sep. 2023.

[9] P. Cunningham and S. J. Delany, "K-nearest neighbour classifiers—a tutorial," *ACM Computing Surveys*, vol. 54, no. 6, pp. 1–25, 2022.

[10] R. Wei, S. Xu, L. Zhong, Z. Yang, Q. Guo, Y. Wang, R. Wang, and M. Li, "LightMamba: Efficient mamba acceleration on FPGA with quantization and hardware co-design," in *2025 Design, Automation & Test in Europe Conference (DATE)*, 2025.

[11] A. Wang, H. Shao, S. Ma, and Z. Wang, "FastMamba: A high-speed and efficient mamba accelerator on FPGA with accurate quantization," 2025. [Online]. Available: https://arxiv.org/abs/2505.18975

[12] X. Jin, H. Zheng, M. Nie, J. Wang, and C. R. Shi, "HCSAs: Hybrid computing systolic arrays for accelerating mamba models with unified state space buffers and energy-efficient dataflow," in *Proceedings of the Great Lakes Symposium on VLSI 2025*. New York, NY, USA: ACM, 2025, pp. 457–462.

[13] I. Yaman, G. Tian, M. Larsson, P. Persson, M. Sandra, A. Dürr, E. Tegler, N. Challa, H. Garde, F. Tufvesson, K. Åström, O. Edfors, S. Malkowsky, and L. Liu, "The LuViRA dataset: Synchronized vision, radio, and audio sensors for indoor localization," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 11 920–11 926.

[14] S. Malkowsky, J. Vieira, L. Liu, P. Harris, K. Nieman, N. Kundargi, I. C. Wong, F. Tufvesson, V. Öwall, and O. Edfors, "The world's first real-time testbed for massive MIMO: Design, implementation, and validation," *IEEE Access*, vol. 5, pp. 9073–9088, 2017.

[15] I. Yaman, G. Tian, E. Tegler, J. Gulin, N. Challa, F. Tufvesson, O. Edfors, K. Åström, S. Malkowsky, and L. Liu, "LuViRA dataset validation and discussion: Comparing vision, radio, and audio sensors for indoor localization," *IEEE Journal of Indoor and Seamless Positioning and Navigation*, vol. 2, pp. 240–250, 2024.

[16] C. Zeng, Z. Liu, G. Zheng, and L. Kong, "CMamba: Channel correlation enhanced state space models for multivariate time series forecasting," 2024. [Online]. Available: https://arxiv.org/abs/2406.05316

[17] T. Sutikno, A. Z. Jidin, A. Jidin, and N. R. N. Idris, "Strategies for FPGA implementation of non-restoring square root algorithm," *International Journal of Electrical and Computer Engineering*, vol. 4, no. 4, pp. 548–556, 2014.

[18] M. Istoan and B. Pasca, "Fixed-point implementations of the reciprocal, square root and reciprocal square root functions," 2015. [Online]. Available: https://hal.science/hal-01229538